



Rapport de stage de fin d'année

Encadré par:
M. Perisse Thierry

Fait par:
M. Ismail Haddad
M. Abderaouf Rehali

Parcours universitaire:
Master 1 Systèmes et Microsystèmes Embarqués

Année universitaire 2024/2025

Remerciements

*Nous remercions tout particulièrement **M. Thierry Périssé**, notre tuteur, pour avoir mis à notre disposition le matériel et la documentation nécessaires, ainsi que pour ses conseils avisés qui ont guidé l'avancement de ce projet. Nous souhaitons également souligner l'efficacité et la complémentarité du travail en binôme, qui a permis de mener à bien cette étude dans de bonnes conditions.*

Table de matieres

I . Introduction.....	4
II . Présentation du système VMD et du module CAN01A.....	5
II .1. Le système VMD : un véhicule multiplexé didactique.....	5
II .2. Le module CAN01A : sous-système dédié à la signalisation lumineuse.....	6
III. Présentation de l'architecture Arduino utilisée pour remplacer l'EID210.....	9
III.1. Contexte et objectif.....	10
III.2. Architecture matérielle.....	10
III.3. Organisation logicielle.....	16
III.4. Avantages de l'architecture Arduino.....	17
IV. Communication CAN avec le système VMD.....	17
IV.1. Présentation du protocole CAN (Controller Area Network).....	17
IV.2. Principe de Communication du Bus CAN.....	20
IV.3. Types de Bus CAN.....	23
IV.4. Architecture Physique du Bus CAN.....	24
IV.5. Composants clés utilisés (MCP2515 et MCP25050).....	24
IV.5.1. MCP2515 (contrôleur CAN):.....	24
A- Schéma de liaisons principales SPI entre l'arduino et l'extension shield CAN-BUS :	24
B- Schéma de câblage entre le buzzer et l'extension shield CAN-BUS :	25
C- Fonctionnement de la communication SPI (CS, MOSI, MISO, SCK) avec interruption:	26
D- Séquence du flux SPI:.....	29
IV.5.2. MCP25050 (module esclave du CAN01A).....	29
IV.5.3. Horloge et vitesse de communication.....	31
V. Implémentation logicielle sur Arduino.....	32
V.1. Organisation générale du code Arduino.....	32
V.2. Fonctionnement de l'interruption matérielle.....	33
V.3. Décodage et envoi des trames.....	33
VI. Réalisation des travaux pratiques avec Arduino.....	34
VI.1. TP1 : Faire commuter les lampes d'un bloc optique.....	35
VI.2. TP2 : Acquérir l'état du commodo feux.....	45
VI.3. TP3 : Vérifier le fonctionnement d'un bloc optique.....	52
VI.4. TP4 : Commander feux a partir du commodo feux.....	58
VII. Comparaison entre Arduino Uno R3 et EID210.....	64
VIII. ANNEXE.....	65
1. Documents techniques et constructeurs:.....	66
2. Documents pédagogiques et manuels du VMD:.....	66
3. Sources externes utilisées pour enrichir certaines parties:.....	66

I . Introduction

Le protocole **CAN** (Controller Area Network) est un standard de communication série largement adopté dans les systèmes embarqués. Conçu pour fonctionner sans contrôleur central, il permet à plusieurs nœuds (microcontrôleurs, capteurs, actionneurs) d'échanger des données de manière décentralisée grâce à une architecture multi-maître. Les messages sont transmis via des trames prioritaires, assurant une communication fiable et efficace. Le développement des systèmes embarqués et la généralisation des protocoles de communication tels que le CAN ont profondément transformé les architectures électroniques dans les secteurs de l'automobile, de l'aéronautique, et plus largement des systèmes industriels. Le **CAN** permet une communication fiable entre plusieurs nœuds (capteurs, actionneurs, calculateurs) à travers un bus unique, réduisant ainsi la complexité du câblage tout en augmentant la robustesse et la flexibilité des systèmes. L'apprentissage de ces technologies est devenu un élément essentiel de la formation des ingénieurs en électronique embarquée et en automatique.

Dans ce contexte, le Véhicule Multiplexé Didactique (*VMD*), constitue une plateforme pédagogique utilisée pour simuler un réseau embarqué de type automobile. Traditionnellement, le contrôle du *VMD* était réalisé à l'aide d'une carte spécialisée nommée *EID210*, équipée d'un microprocesseur 32 bits *Motorola 68332* et d'un environnement de développement propriétaire. Bien que performant, ce système présente plusieurs inconvénients : complexité de programmation en assembleur/C bas-niveau, dépendance à un environnement matériel spécifique (bus PC/104), coût élevé et manque de flexibilité pour des développements rapides.

Afin de moderniser cette plateforme et de faciliter l'accès aux expérimentations, le projet mené dans le cadre de ce stage consiste à remplacer la carte *EID210* par une solution basée sur un microcontrôleur Arduino Uno R3, couplée à un module CAN (*MCP2515*) via le *CAN-BUS Shield v1.1 de Seeed Studio*. Cette alternative, économique et largement répandue dans l'écosystème maker, permet de réimplémenter l'ensemble des Travaux Pratiques initialement conçus pour l'*EID210* en utilisant un langage plus accessible (Arduino C++) et des outils modernes.

L'objectif principal du stage est donc de garantir le bon fonctionnement de tous les modules du *VMD* via Arduino, en assurant une compatibilité totale avec les TP1 à TP4 déjà existants :

- **TP1** : Faire commuter les lampes d'un bloc optique (chenillard),
- **TP2** : Acquérir l'état du commodo feux
- **TP3** : Vérifier le fonctionnement d'un bloc optique
- **TP4** : Commander feux a partir du commodo feux

Au-delà de l'aspect technique, ce projet permet également d'acquérir des compétences concrètes en :

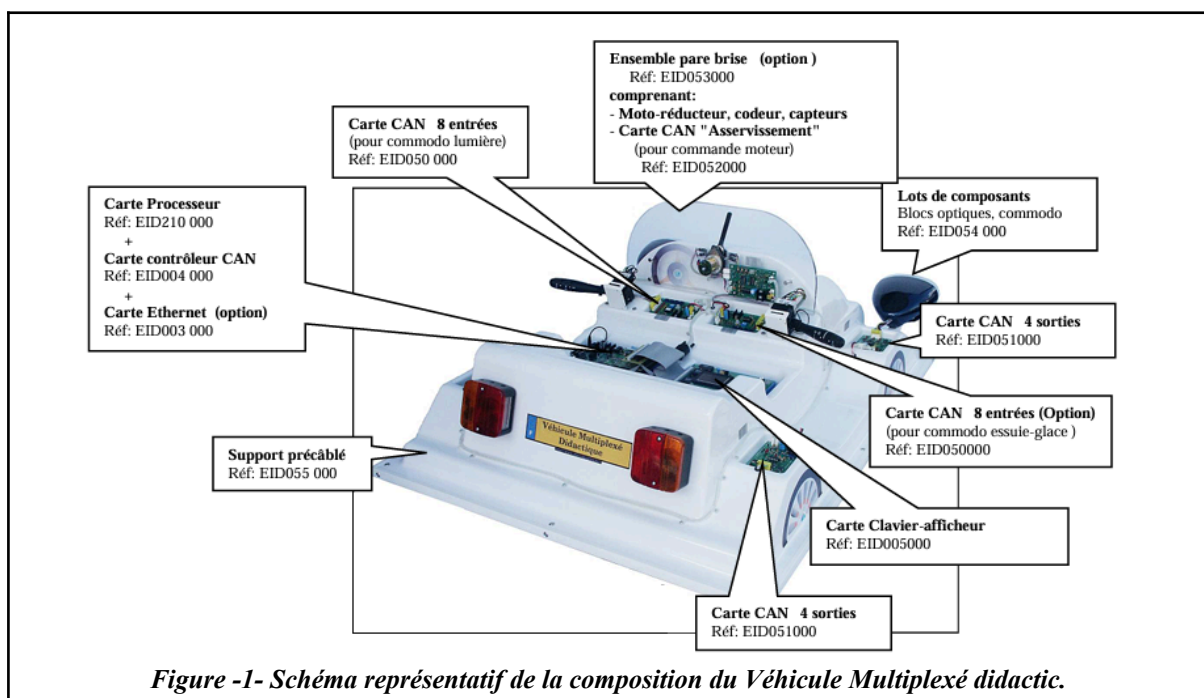
- Programmation orientée microcontrôleur.
- Gestion du bus CAN via des trames de type IM, OM, IRM, AIM.
- Analyse de signaux via oscilloscope (*Picoscope*).
- utilisation d'interruptions matérielles,
- Communication SPI entre contrôleur et shield CAN.

II . Présentation du système VMD et du module CAN01A

II.1. Le système VMD : un véhicule multiplexé didactique

Le *VMD (Véhicule Multiplexé Didactique)* est une maquette pédagogique conçue pour simuler le fonctionnement d'un système embarqué automobile réel, tout en permettant une accessibilité complète aux signaux, à la logique de contrôle, aux bus de communication, et aux unités fonctionnelles. Il est couramment utilisé dans les contextes d'automatisation, et aux protocoles de communication comme le **CAN (Controller Area Network)**.

Ce système est structuré autour de plusieurs modules répartis selon différentes fonctions du véhicule : gestion moteur, feux de signalisation, capteurs et actionneurs divers, interfaces de commande (comme le commodo), etc. Chaque module est interconnecté via un bus CAN, ce qui permet une architecture décentralisée, semblable aux architectures utilisées dans les véhicules automobiles modernes. Le *VMD* sert donc d'environnement idéal pour la mise en œuvre concrète des protocoles industriels et des microcontrôleurs spécifiques comme ceux de Microchip (*MCP2515*, *MCP25050*...).

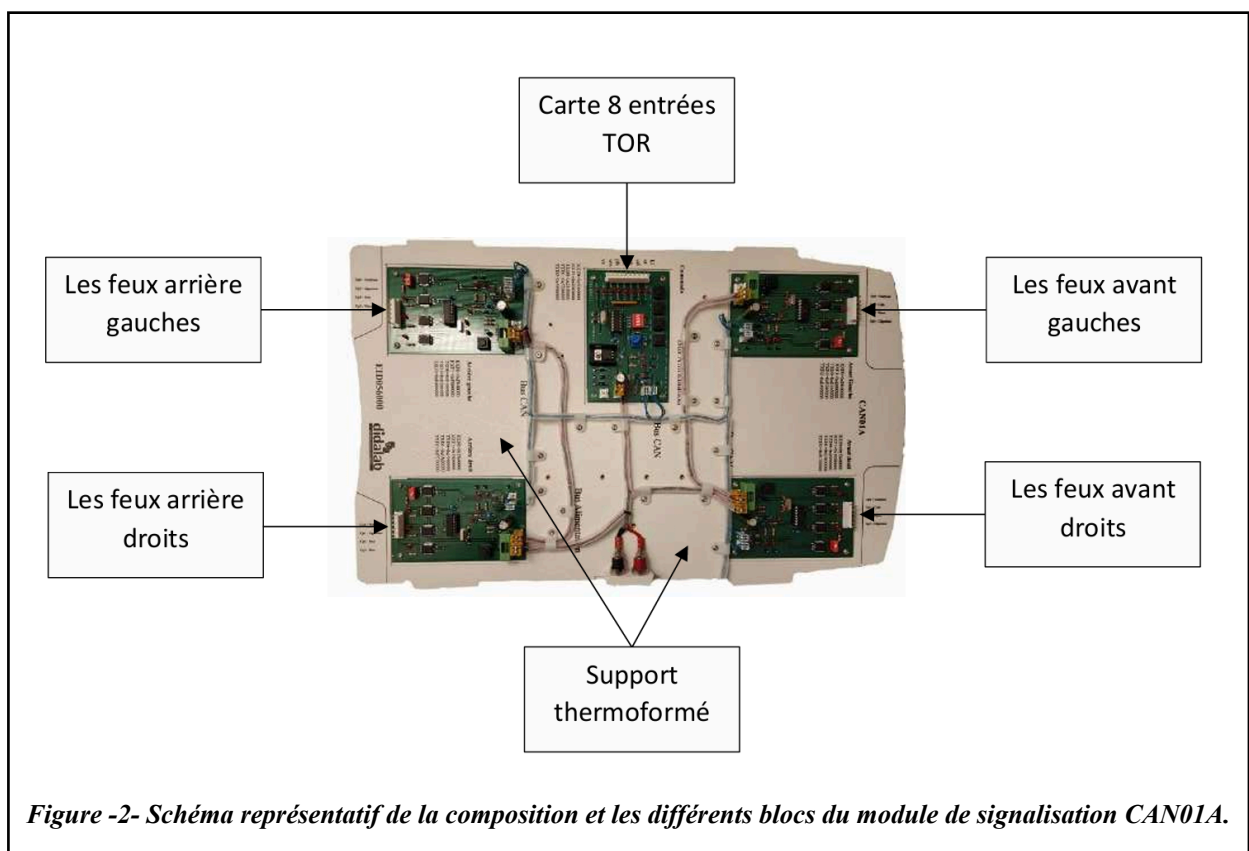


Cette modularité permet d'effectuer des TP (travaux pratiques) orientés vers des cas d'usage concrets : lecture d'état du commodo, commande des feux, interrogation de capteurs, analyse de trames **CAN**, et débogage avec des outils comme un oscilloscope ou un analyseur logique. Le *VMD* constitue un véritable banc de simulation automobile à échelle réduite, tout en offrant un accès direct aux éléments internes pour des manipulations pratiques.

II.2. Le module **CAN01A** : sous-système dédié à la signalisation lumineuse

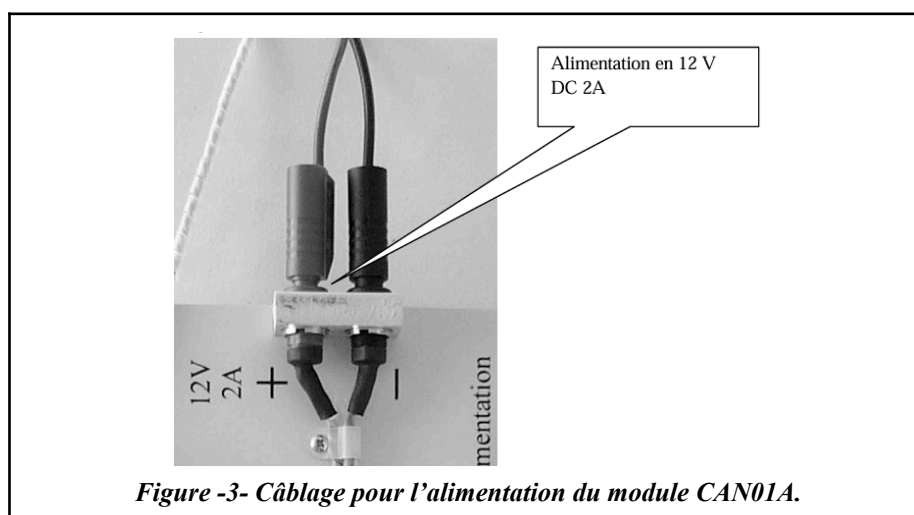
Le module **CAN01A** est l'un des sous-ensembles fonctionnels du *VMD*, dédié spécifiquement à la gestion des feux de signalisation : feux avant (code, phare, veilleuse), feux arrière (stop, veilleuse), clignotants, ainsi que le klaxon. Ce module fonctionne en interaction étroite avec le « commodo » (interface utilisateur simulée), dont il reçoit les ordres via des trames **CAN**. Ce sous système est constitué de plusieurs blocs connectés sur un bus **CAN** :

- Une carte 8 entrées TOR sur bus **CAN** gérant le commodo lumière
- 4 cartes de sortie TOR, gérant :
 - Les feux avant gauche.
 - Les feux avant droit.
 - Les feux arrière gauche.
 - Les feux arrière droit.



Techniquement, le CAN01A repose sur un circuit intégré Microchip *MCP25050*, une interface CAN embarquant à la fois une transceiver CAN (réf:82CA251) et une logique d'E/S configurable. Ce circuit gère de manière autonome la lecture des entrées (boutons du commodo) et la commande des sorties (LED simulant les feux) à travers des registres internes accessibles via des trames de type IM (*Instruction Message*), IRM (*Information Request Message*) ou OM (*Output Message*).

Le sous système CAN01A est alimenté en +12 V DC

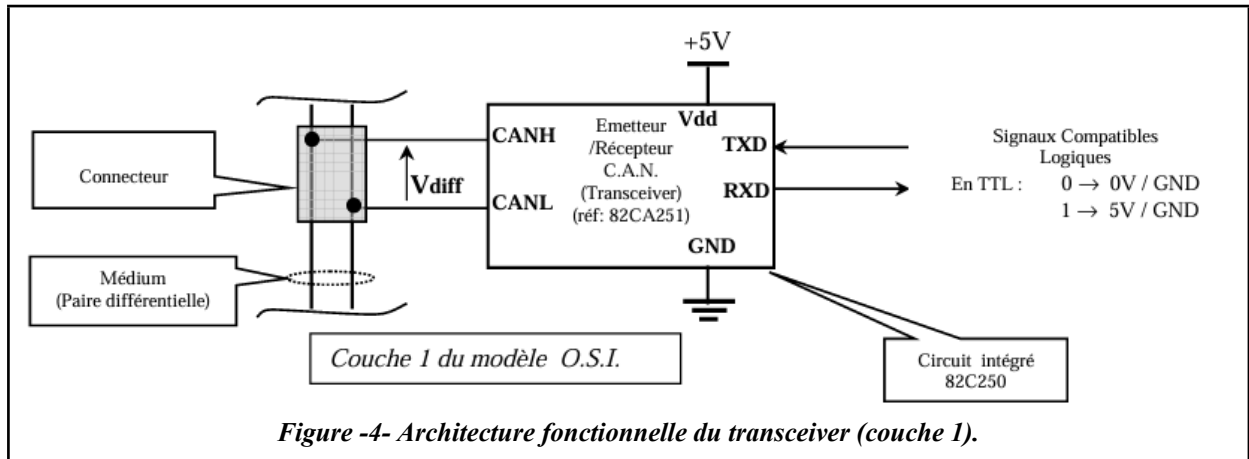


Ce module est donc essentiel dans la gestion de la signalisation lumineuse du *VMD*. Il offre :

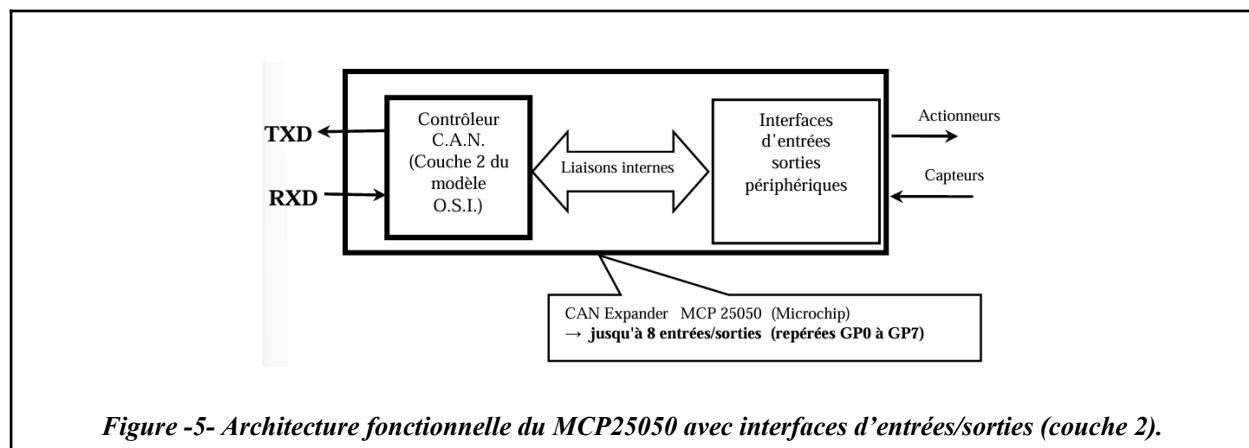
- Une capacité de configuration des broches (en entrée ou sortie) via les registres GPDDR.
- La possibilité de lire les états logiques des broches via GPLAT.
- L'émission automatique de trames OM en cas de changement d'état.
- La possibilité d'être commandée à distance via des trames IM.

Tous ces modules comportent les éléments suivants :

- Deux connecteurs de liaison au médium (fils de liaisons bus CAN) permettant de connecter les modules en série.
- Deux connecteurs d'alimentation (bus "alimentation énergie").
- Un régulateur de tension générant la tension +5V nécessaire aux circuits intégrés inclus sur le module ainsi que les protections d'usage et une LED de visualisation de présence tension.
- Une résistance de terminaison de bus connectable ou non par "jumper".
- Un circuit intégré émetteur récepteur de ligne (réf: 82CA251) réalisant l'interface électrique (mode différentiel côté bus CAN et mode référencé côté application), c'est-à-dire la couche 1 du modèle O.S.I.



- un circuit intégré d'extension d'entrées sorties (Réf: *MPC25050*) permettant la communication et l'interprétation des messages, c'est-à-dire la couche 2 du modèle O.S.I.
- Un oscillateur à quartz est nécessaire au circuit *MPC25050*.
- Une LED de visualisation de communication en réception.
- Une LED de visualisation de communication en émission.



Dans le cadre du projet de remplacement de la carte *EID210* par une carte Arduino Uno R3 associée à un shield **CAN**, le module **CAN01A** représente le point central de test et d'expérimentation. Il a servi à valider les communications **CAN**, à simuler des scénarios de conduite (activation des feux selon les boutons du commodo), et à assurer un fonctionnement conforme aux standards automobiles.

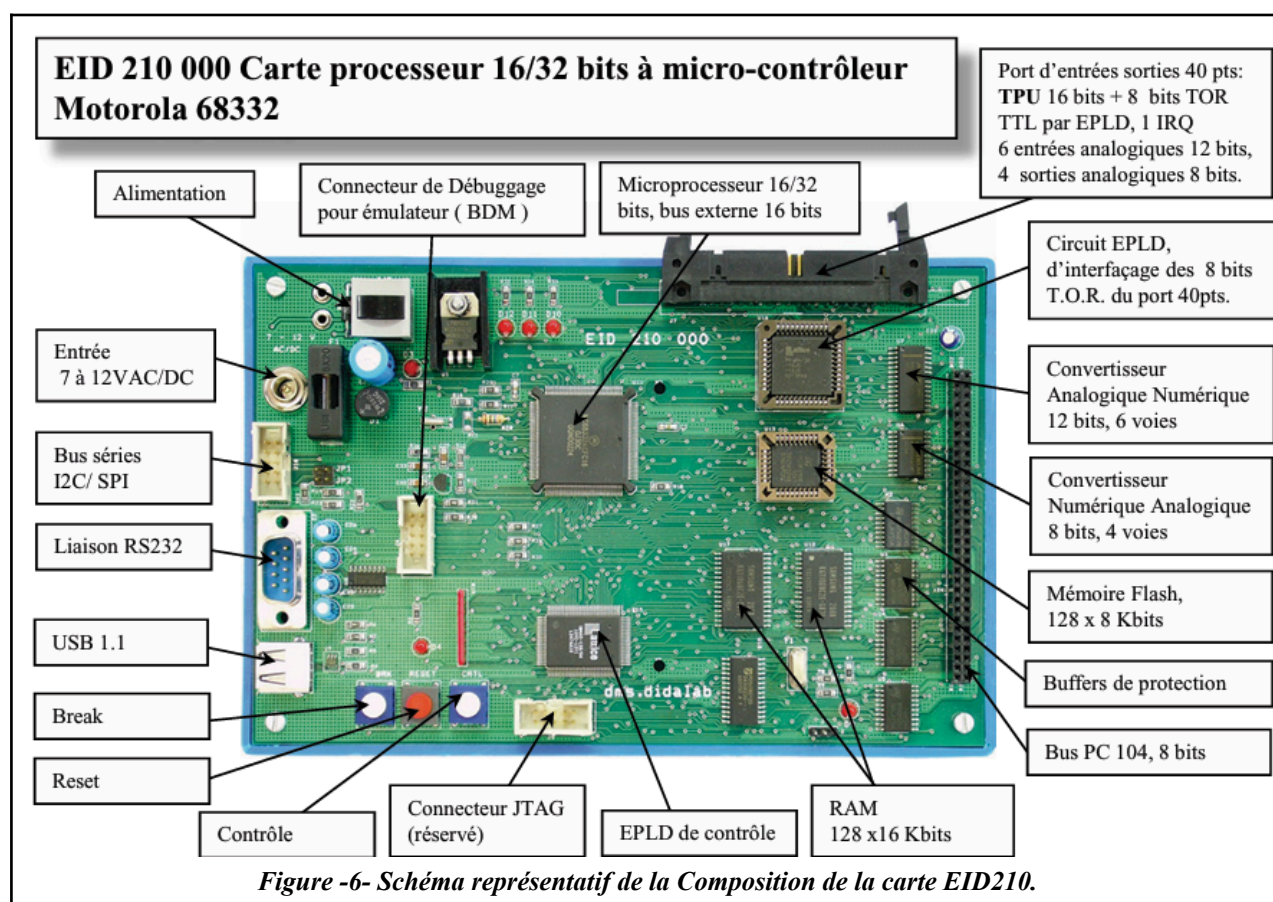
III. Présentation de l'architecture Arduino utilisée pour remplacer l'EID210

III.1. Contexte et objectif

La carte *EID210* est précédemment utilisée dans le banc pédagogique *VMD* (Véhicule Multiplexé Didactisé) a été remplacée par une carte Arduino Uno R3 équipée d'un CAN-BUS Shield v1.1 de Seeed Studio. Ce remplacement vise à moderniser la plateforme en se basant sur une architecture plus accessible, open-source, facile à programmer, et économiquement avantageuse.

La carte *EID210* est une plateforme matérielle spécifique, souvent fournie avec des bibliothèques propriétaires et des interfaces logicielles spécifiques à l'environnement Didalab. Bien que robuste, cette solution manque de souplesse, surtout pour des applications évolutives.

Le remplacement par un système Arduino + *MCP2515* permet d'interfacer directement avec le sous-système CAN01A, dédié à la commande des feux de signalisation (feux avant, feux arrière, commodo, etc.), tout en implémentant les TP (TP1 à TP4) initialement conçus pour la carte *EID210*.



III.2. Architecture matérielle

L'architecture proposée se base sur les composants suivants :

Arduino Uno R3: C'est une carte de développement à microcontrôleur basée sur le *ATmega328P*. Elle est largement utilisée dans les domaines de l'embarqué, de la robotique et de la domotique grâce à sa simplicité, sa communauté active et sa compatibilité avec une vaste bibliothèque d'extensions (notamment *mcp2515.h* pour le CAN). Elle dispose d'entrées/sorties numériques, d'interfaces de communication (SPI, I2C, UART), et permet un prototypage rapide en langage C/C++ via l'environnement Arduino IDE.

Dans le projet, l'Arduino Uno agit comme maître du système : il lit les trames CAN émises par les périphériques, commande dynamiquement les feux, active ou désactive des actionneurs (comme le buzzer), et gère l'affichage d'informations sur le moniteur série. Grâce à sa simplicité d'usage, sa stabilité, l'Arduino Uno constitue une plateforme robuste et pédagogique pour les systèmes embarqués en environnement multiplexé.

Caractéristiques principales:

- Microcontrôleur : *ATmega328P* (8 bits).
- Fréquence d'horloge : 16 MHz.
- Mémoire Flash : 32 KB (dont 0.5 KB utilisés par le bootloader).
- RAM : 2 KB.
- EEPROM : 1 KB.
- Tensions : 5 V (logique), 7–12 V (entrée).
- 14 broches numériques (dont 6 PWM).
- 6 entrées analogiques.
- Interface SPI, I2C, UART.
- Broche d'interruption externe utilisée : D2 (INT).

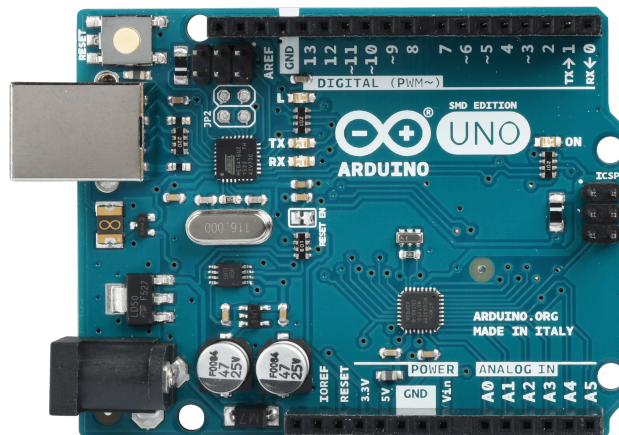


Figure -7- Carte Arduino Uno R3, face avant ([UNO R3 | Arduino Documentation](#)).

CAN-BUS Shield v1.1: C'est une extension matérielle pour Arduino permettant d'interfacer la carte avec un réseau CAN (Controller Area Network). Il est basé sur deux composants principaux : le *MCP2515*, contrôleur CAN SPI, et le *MCP2551*, transceiver physique qui assure la conversion des signaux logiques en signaux différentiels compatibles avec le bus CAN. Ce shield permet à un Arduino de s'insérer dans un réseau de communication industriel ou automobile.

Dans le projet, il est utilisé pour échanger des trames CAN avec le commodo et les modules feux du *VMD*. Il reçoit les trames OM (*Output Message*) pour lire l'état des boutons du commodo, et envoie des trames IM (*Instruction Message*) pour piloter les feux et le klaxon. La broche INT est utilisée pour générer des interruptions sur l'Arduino (D2), garantissant une réactivité optimale

Caractéristiques principales:

1. **Interface DB9:** Permet de se connecter à une interface OBDII via un câble DBG-OBDD.
2. **V_OBD:** Alimentation reçue depuis l'interface OBDII (par la prise DB9).
3. **Voyants LED:**
 - **PWR** : alimentation.
 - **TX** : clignote lors de l'envoi de données.
 - **RX** : clignote lors de la réception de données.
 - **INT** : indique une interruption de données.
4. Bornier à vis pour connexion CAN_H / CAN_L.
5. Sorties Arduino UNO (Alimentation 5 V directement via Arduino).
6. Connecteur Grove série.
7. Connecteur Grove I2C.
8. Broches ICSP.
9. **Transceiver CAN haute vitesse : MCP2551** (Circuit intégré).
10. **Contrôleur CAN : MCP2515** (Interface SPI jusqu'à 10 MHz) (Circuit intégré).
 - Implémente CAN V2.0B jusqu'à 1 Mb/s.
 - Interface SPI avec l'Arduino (CS = D9 par défaut).

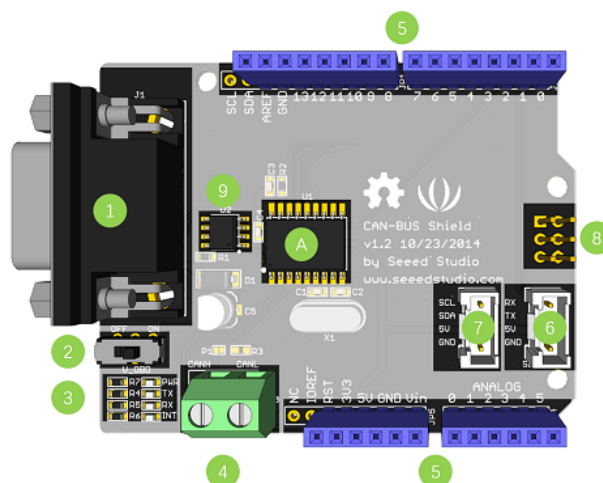


Figure -8- Carte CAN-BUS, face avant ([CAN-BUS Shield V1.2 | Seesd Studio Wiki](#)).

PicoScope 3204D MSO: C'est un oscilloscope numérique portable à mémoire (DSO) avec fonctionnalités mixtes analogiques et logiques (MSO : Mixed-Signal Oscilloscope). Il se connecte à un ordinateur via USB et offre des fonctionnalités avancées telles que les déclenchements conditionnels, le décodage de protocoles série (CAN, I2C, SPI), ou l'enregistrement longue durée. Il est particulièrement adapté pour le débogage de signaux numériques rapides dans des environnements embarqués.

Dans ce projet, le *PicoScope* a servi à visualiser les trames CAN ou SPI en temps réel (niveau physique), et à identifier des anomalies.

Caractéristiques principales:

- Canaux analogiques : 2.
- Canaux logiques (MSO) : 16.
- Bande passante : 70 MHz.
- Échantillonnage max : 1 GS/s (analogique), 80 MS/s (logique).
- Résolution : 8 bits à 1 GS/s.
- Mémoire tampon : 64 MS.
- Interface : USB 3.0 (alimentation et données).
- Décodeurs intégrés : CAN, UART, SPI, etc.



Figure -9- Picoscope 3204D ([PicoScope 3000 Series USB3.0 oscilloscope specifications](#)).

Buzzer V1.2: C'est un buzzer actif, c'est-à-dire qu'il émet un son de fréquence fixe (environ 2 kHz) lorsqu'un signal numérique haut lui est appliqué. Ce module est simple à utiliser, compact, et compatible avec les cartes Arduino via le connecteur Grove (ou en câblage classique avec VCC, GND et SIG). Il est souvent utilisé pour générer des alertes sonores ou des retours utilisateurs dans des systèmes embarqués.

Dans le cadre du projet, le buzzer est relié à la broche D3 de l'Arduino. Il est activé en temps réel lors de l'appui sur le bouton klaxon (GP7) du commodo, détecté via une trame CAN.

Caractéristiques principales:

- Type : Buzzer actif (fréquence fixe).
- Tension de fonctionnement : 3.3 V à 5 V.
- Broches :
 - **SIG** : signal d'activation (connectée à une sortie numérique)
 - **VCC**
 - GND
 - NC : non utilisée
- Consommation : faible (adapté à l'IoT).
- Fréquence typique : ~2 kHz.

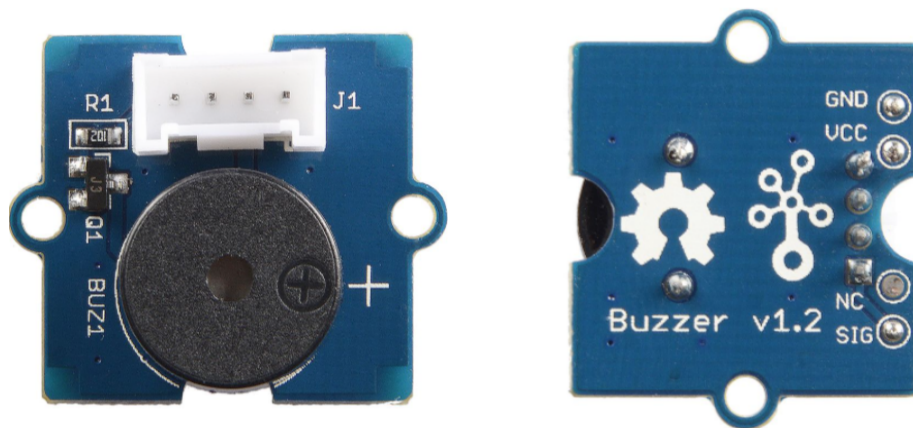


Figure -10- Buzzer by seeed studio ([Grove - Buzzer | Seeed Studio Wiki](#)).

Modules VMD (CAN01A): Le système *VMD* intègre plusieurs sous-modules connectés via un réseau CAN. Le CAN01A est le module principal utilisé ici pour la signalisation lumineuse, incluant les feux avant, les feux arrière et le commodo.

- Feux avant/arrière (gauche/droit) : Ces modules sont chacun pilotés par un circuit *MCP25050*, un contrôleur CAN avec fonctions d'E/S intégrées. Ils reçoivent des trames de type IM contenant les instructions d'allumage (veilleuse, phare, clignotants, etc.) et activent les LED correspondantes via leurs sorties GP. Ces blocs comprennent les éléments suivants:
 - 4 LEDs de visualisation des états des sorties puissance.
 - 4 entrées de contrôle des charges.
 - 4 interfaces de puissance permettant de piloter 4 charges électriques en "tout ou rien", sous 12V (VN05).
 - 1 entrée de simulation de coupure de charge.

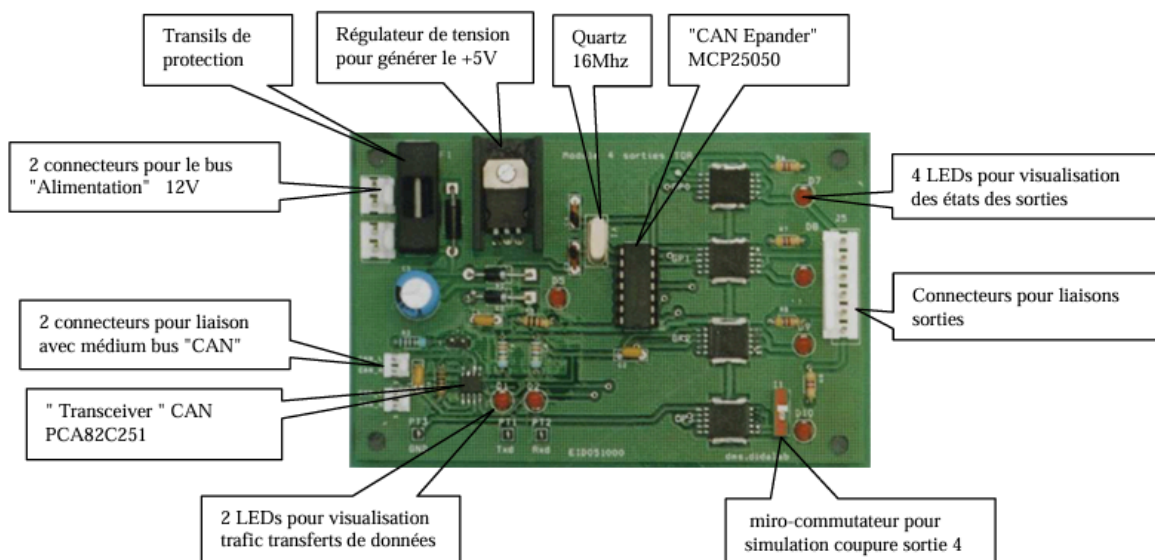


Figure -11- Schéma représentatif de la composition du bloc optique à 4 sorties du module CAN01A.

Le VN05 agit à la fois comme un détecteur de charge (via son signal logique) et comme un pilote de puissance sécurisé, capable de gérer des courants élevés tout en évitant les pannes dues aux surcharges, Il a deux rôles principaux :

1. **Détection de la charge connectée:** Le VN05 mesure le courant consommé par la charge (ex. : la led) et génère un signal logique (0/1) indiquant si elle est bien branchée. Ce signal permet au système de vérifier son bon fonctionnement (utile pour un *VMD*).
2. **Gestion et protection de la puissance:** Le VN05 alimente des charges jusqu'à 12 A et possède des protections intégrées (court-circuit, surchauffe), assurant une commande fiable et sécurisée.

- Commodo (CAN01A) : Équipé également d'un *MCP25050*, ce module sert d'interface utilisateur. Il convertit les appuis sur les boutons physiques (GP0 à GP7) en trames OM (*Output Message*) diffusées automatiquement sur le bus CAN.

ce module comprend :

- 4 boutons poussoir
- 4 commutateurs à 2 positions (fermé ou ouvert)
- 1 connecteur 10 points permettant de relier les capteurs externes de type fin de course
- 8 LEDs de visualisation des états
- 1 potentiomètre analogique.

Chaque bouton (veilleuse, klaxon, clignotants, stop, etc.) est assigné à un bit d'un registre GPx.

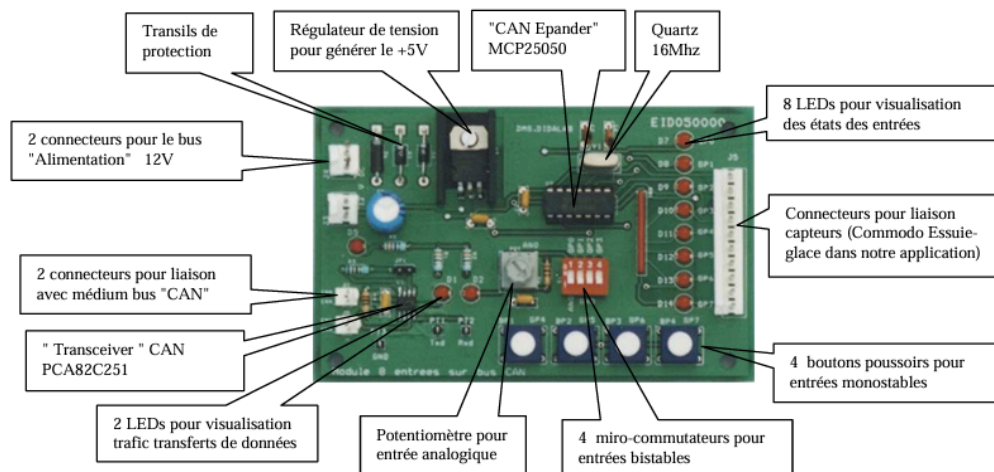


Figure -12- Schéma représentatif de la composition du bloc commodo à 8 entrées du module CAN01A.

Une LED s'allume si l'entrée correspondante est forcée à 0 (commutateur en position "fermé" ou bouton poussoir "appuyé")

III.3. Organisation logicielle

L'architecture logicielle repose sur la bibliothèque *mcp2515.h* (open-source), permettant une gestion fine du contrôleur *MCP2515* : configuration, lecture/écriture de trames CAN, gestion des interruptions, etc.

Le code Arduino implémente :

- La configuration initiale des modules (IM pour régler GPDDR et IOTEN).
- L'interrogation cyclique ou interruptive du commodo (via trames IRM, écoute passive des OM).
- Le traitement des trames reçues et la mise à jour des états logiques.
- L'émission des commandes IM vers les blocs feux (avant gauche, avant droit, arrière gauche, arrière droit).
- Une visualisation série pour le suivi pédagogique et le débogage.
- L'intégration d'un buzzer Grove sur le pin D3, pour simuler le klaxon lors de l'activation du bouton GP7 du commodo.

III.4. Avantages de l'architecture Arduino

L'adoption de l'architecture Arduino Uno R3 offre de nombreux avantages, notamment dans un cadre expérimental. Grâce à son environnement de développement simple et accessible, Arduino permet une prise en main rapide, aussi bien pour l'écriture du code que pour la mise en œuvre matérielle. Contrairement à la carte *EID210*, qui repose sur un environnement propriétaire plus rigide, l'Arduino permet une grande souplesse de programmation, facilitant l'expérimentation et le débogage. Le CAN-BUS Shield, équipé du contrôleur *MCP2515* et du transceiver *MCP2551*, assure une compatibilité totale avec le protocole CAN 2.0B utilisé par les modules du *VMD*. Cette architecture permet une interopérabilité facile avec d'autres composants comme un buzzer ou des capteurs.

IV. Communication CAN avec le système VMD

IV.1. Présentation du protocole CAN (Controller Area Network)

Le bus CAN est un protocole de communication série conçu initialement par Bosch dans les années 1980 pour l'automobile, afin de permettre à différents calculateurs électroniques (ECU) d'échanger des données de manière fiable, rapide et sans maître central. Il repose sur une topologie en bus, avec deux lignes différentielles (CAN_H et CAN_L), sur lesquelles tous les nœuds (appelés nœuds CAN) sont connectés en parallèle.

Dans le cadre du système *VMD*, le CAN permet la communication entre plusieurs modules embarqués (comme le commodo, les blocs optiques avant/arrière, etc.). Le protocole utilisé

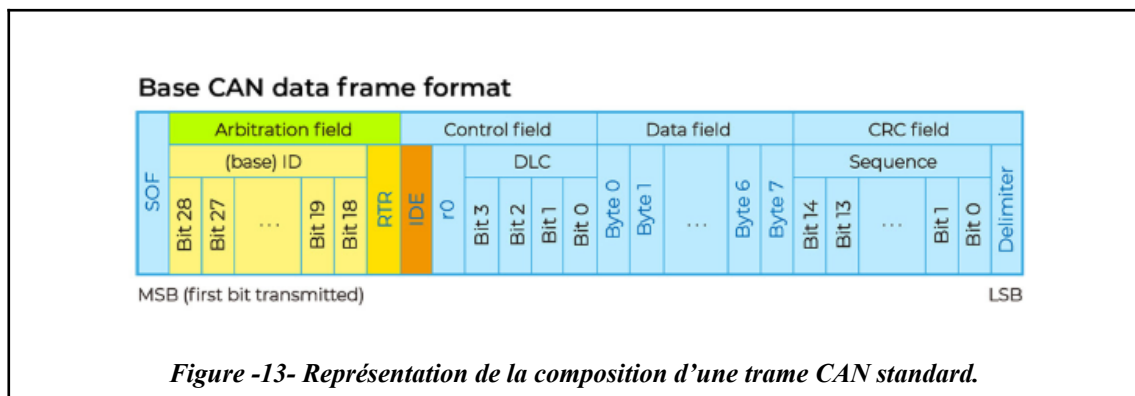
est basé sur le CAN 2.0B, qui prend en charge les identifiants étendus (29 bits), ce qui permet une hiérarchisation fine des trames.

La vitesse de communication utilisée dans le projet est de 100 kbps, adaptée aux besoins du système *VMD*, qui ne nécessite pas un débit élevé mais plutôt une fiabilité et une robustesse. Cette vitesse est suffisante pour l'échange de trames de contrôle entre les modules (feux, commodo, etc.) sans provoquer de saturation du bus.

Il existe deux types de Structure des trames :

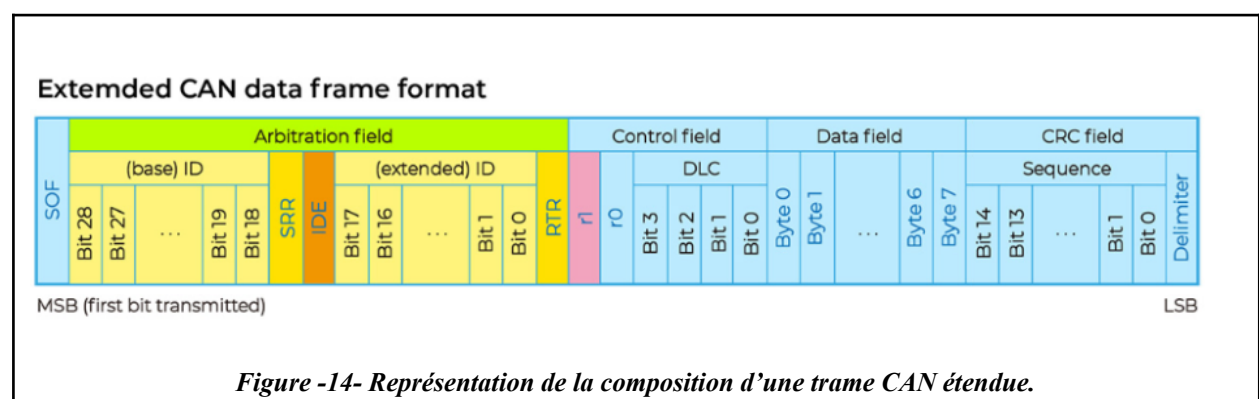
- Trame standard

Elle utilise un identifiant de 11 bits, ce qui permet de gérer jusqu'à 2048 identifiants différents.



- Trame étendue

Elle utilise un identifiant de 29 bits, permettant ainsi une plus grande capacité d'adressage, avec jusqu'à 536 870 912 identifiants possibles.



Champ	Longueur (bits)	Description
SOF	1	Start of Frame : Marque le début de la trame (toujours 0 = dominant)
Champ d'arbitrage :	<u>32</u>	
ID (11 bits)	11	Bits d'identifiant prioritaire (partie haute)
SRR	1	Substitute Remote Request : (1) pour trames étendues
IDE	1	Identifier Extension : il se trouve dans le champ d'arbitrage si la trame est étendue (1) , et dans le champ de contrôle si la trame est standard (0) .
ID (18 bits)	18	Bits d'identifiant secondaires (partie basse)
RTR	1	Remote Transmission Request: détermine s'il s'agit d'une trame de données (0) ou d'une trame d'interrogation (1) (elle ne contient pas de données).
Champ de contrôle:	<u>6</u>	
r1 & r0 (reserved)	1 (chacun)	Réservé et ne sont pas utilisés (dominant : 0)
DLC	4	Data Length Code : nombre d'octets de données (0 à 8)
Champ de données:		
Data	0 à 64	Données utiles (jusqu'à 8 octets en CAN 2.0B)
Champ CRC:	<u>16</u>	
CRC (15 bits)	15	Code de redondance cyclique (permet de vérifier l'intégrité des données reçues).

CRC Delimiter	1	Toujours récessif (1)
Champ d'acquittement:	2	
ACK Slot	1	Bit d'acquittement attendu à 0 si la trame est reçu correctement
ACK Delimiter	1	Toujours récessif (1)
EOF	7	End of Frame : toujours 1 (récessif)

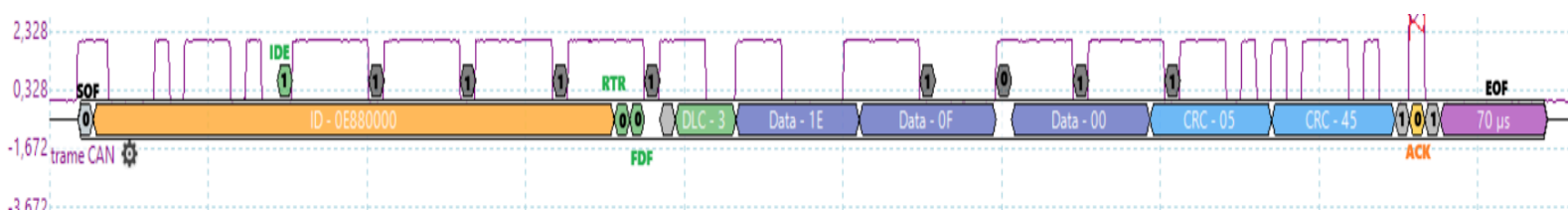


Figure -15- visualisation d'une trame can envoyée depuis le CAN-BUS shield sur le picoscope.

Lors de la réception d'une trame, le récepteur effectue un calcul CRC et compare le résultat avec celui contenu dans la trame. Si les deux résultats correspondent, cela signifie que la trame est correcte.

Le récepteur envoie un signal ACK pour indiquer qu'il a bien reçu la trame. Si aucune confirmation n'est reçue, la trame est renvoyée.

Chaque module *VMD* (comme le CAN01A) publie automatiquement ses états via des trames OM (*Output Message*) et accepte des commandes via des trames IM (*Input Message*). Ce mode de fonctionnement basé sur le CAN assure une communication rapide, déterministe, et tolérante aux erreurs.

IV.2. Principe de Communication du Bus CAN

Le Bus CAN est un protocole de communication différentiel et asynchrone. Ses caractéristiques principales incluent :

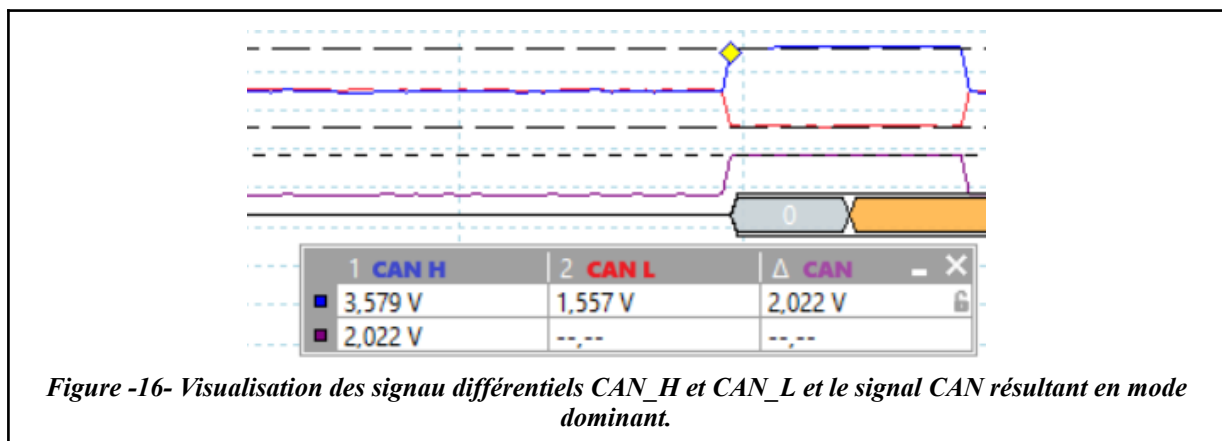
- **Transmission Différentielle du Bus CAN**

Le Bus CAN utilise une communication différentielle pour garantir une transmission robuste, même dans des environnements électriquement bruyants (moteurs, usines). Cette méthode repose sur :

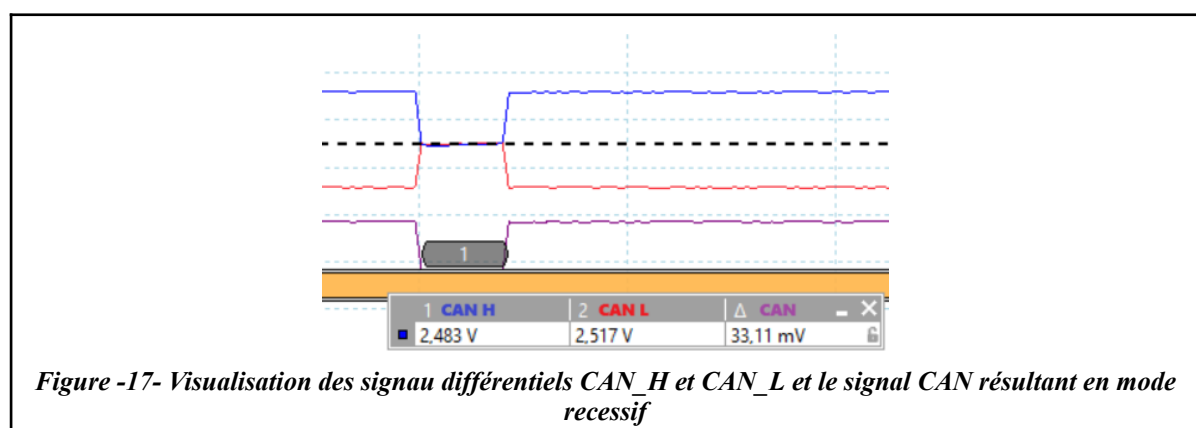
1. **Deux lignes symétriques** : CAN_H (haute tension) et CAN_L (basse tension).

2. Un signal codé par différence de tension entre les deux fils, plutôt que par une tension absolue.
3. Immunité aux interférences : Un bruit électromagnétique affecte les deux fils de manière similaire, mais la différence ($CAN_H - CAN_L$) reste stable.

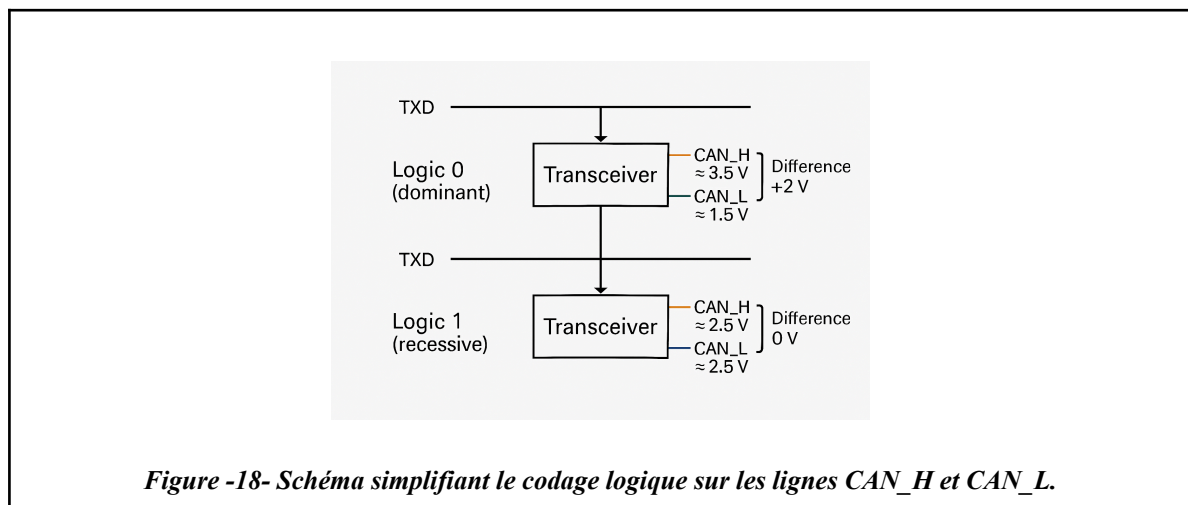
Dans un système CAN, le transceiver (ou interface de ligne) est un composant clé qui assure la liaison entre le contrôleur CAN (comme le *MCP2515*) et le bus physique (les lignes CAN_H et CAN_L). À ce moment-là, le transceiver CAN connecté au *MCP2515* génère généralement un *MCP2551* prend le relais. Son rôle est de convertir les signaux numériques du *MCP2515* TXD, RXD (niveau TTL de 0 V / 5 V) en une différence de tension différentielle sur les lignes CAN_H et CAN_L . Lorsqu'un bit dominant (0) est transmis, le transceiver génère typiquement $CAN_H \approx 3.5\text{ V}$ et $CAN_L \approx 1.5\text{ V}$, soit un écart de $\sim 2\text{ V}$ entre les deux lignes, comme visualisé sur le Picoscope:



Pour un bit récessif (1), les deux lignes sont ramenées à environ 2.5 V, et donc le différentiel devient nul : $CAN_H - CAN_L \approx 0\text{ V}$. Cette modulation différentielle est extrêmement robuste face aux perturbations, ce qui rend le bus CAN si fiable en environnement bruyant.



État logique	CAN_H	CAN_L	Bus CAN
Récessif (1)	~2.5V	~2.5V	Aucune différence → bus au repos
Dominant (0)	~3.5V	~1.5V	Différence de potentiel = communication



Grâce à son principe de fonctionnement basé sur la différence de tension entre CAN_H et CAN_L, le système présente une excellente immunité au bruit électromagnétique - les interférences affectant simultanément les deux lignes sont naturellement filtrées puisque seul l'écart de potentiel est analysé. Cette architecture permet également une détection fiable des pannes: un court-circuit ou une rupture de ligne se traduit immédiatement par une différence de tension anormale (notamment une absence de différentiel), déclenchant les mécanismes de diagnostic intégrés. Ces caractéristiques expliquent pourquoi le **CAN** s'est imposé dans les environnements électriquement hostiles comme l'automobile (norme ISO 11898-2) ou les applications industrielles, où il maintient une communication robuste malgré les fortes perturbations électromagnétiques générées par les moteurs, variateurs de fréquence ou autres équipements haute puissance.

- **Arbitrage basé sur la priorité**

Le **CAN** est un bus multi-maître, plusieurs nœuds peuvent émettre simultanément. En cas de collision, l'identifiant de la trame (plus bas = priorité plus haute) détermine quel message passe en premier (*arbitrage non destructif*).

- **Détection et gestion des erreurs**

Chaque trame intègre un CRC (*Cyclic Redundancy Check*), un ACK (*Acknowledgment*), et d'autres mécanismes. Si une erreur est détectée, la trame est re-transmise automatiquement.

- **Communication asynchrone**

Pas de signal d'horloge global, chaque nœud se synchronise sur les bits de la trame (resynchronisation par *bit stuffing*).

IV.3. Types de Bus CAN

Plusieurs variantes du CAN existent pour s'adapter à différents besoins :

Type	Caractéristiques	Applications
CAN 2.0A (Standard)	Identifiant 11 bits (2^{11} adresses possibles)	Systèmes embarqués basiques
CAN 2.0B (Etendu)	Identifiant 19 bits (compatible avec le 11 bits)	Réseaux complexes
CAN FD	• Débit jusqu'à 8 Mbps (vs 1 Mbps en CAN 2.0). • 64 octets de données (vs 8). • Niveau recessif: CAN_H: 2.5 V CAN_L: 1.5 V • Niveau dominant: CAN_H: 3.5 V CAN_L: 1.5 V •	Véhicules haut débit
CAN High-Speed	1 Mbps max, robuste aux interférences (ISO 11898-2). • Niveau recessif: CAN_H: 2.5 V CAN_L: 2.5 V • Niveau dominant: CAN_H: 3.5 V CAN_L: 1.5 V	Réseaux automobiles (ECU, ABS)

CAN Low-Speed	• 125 kbps max, tolérant aux pannes (ISO 11898-3). • Niveau recessif: CAN_H: 1.75 V CAN_L: 3.25 V • Niveau dominant: CAN_H: 4 V CAN_L: 1 V	Éclairage, confort automobile.
---------------	--	--------------------------------

IV.4. Architecture Physique du Bus CAN

- **Topologie linéaire** : Tous les nœuds sont connectés à une paire torsadée (CAN_H et CAN_L).
- Résistances de terminaison (120 Ω) aux extrémités pour éviter les réflexions de signal.
- **Transceivers CAN** : Convertissent les signaux logiques en tensions différentielles (typ. 2V-3,5V pour CAN_H, 1,5V-2,5V pour CAN_L).
- Communication en broadcast : Chaque message est reçu par tous les nœuds, mais seul le destinataire concerné le traite.

IV.5. Composants clés utilisés (MCP2515 et MCP25050)

IV.5.1. MCP2515 (contrôleur CAN):

Le *MCP2515* est un contrôleur CAN autonome de chez Microchip, utilisé dans le shield CAN-BUS v1.1 pour Arduino. Il gère toute la couche liaison du protocole CAN : transmission, réception, filtrage et mise en file des trames. Il communique avec l'Arduino via le bus SPI (broches D10–D13) et peut générer une interruption via la broche INT (connectée à D2 dans ce projet).

A- Schéma de liaisons principales SPI entre l'arduino et l'extension shield CAN-BUS:

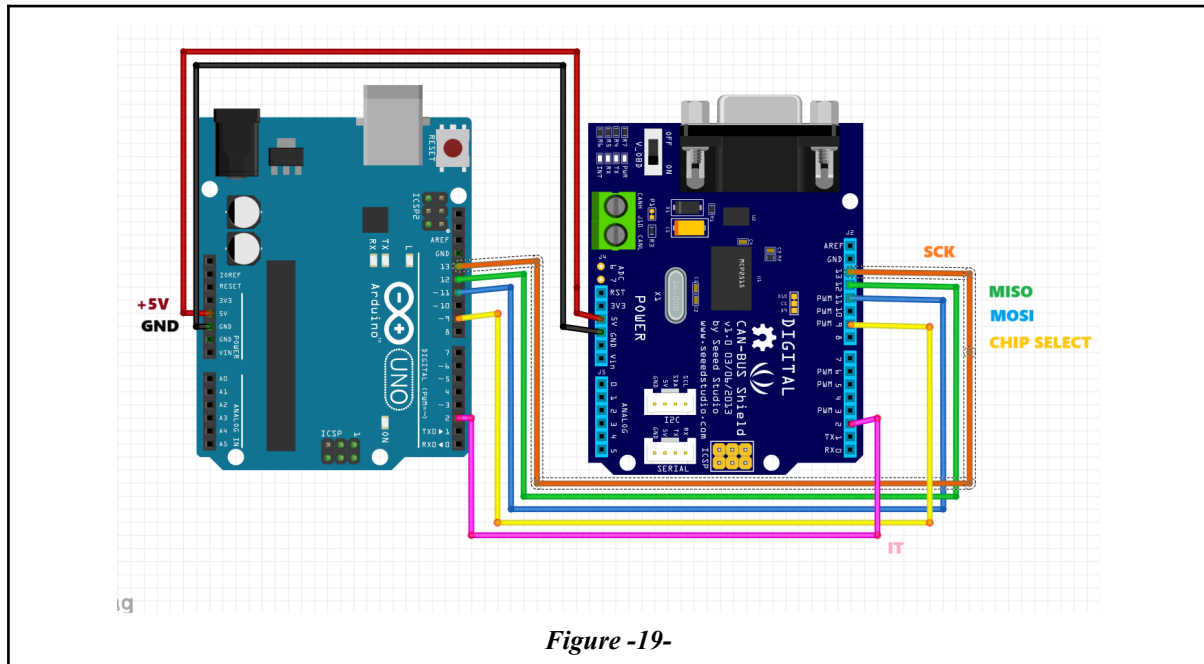


Figure -19-

L'extension se fixe directement sur l'Arduino.

B- Schéma de câblage entre le buzzer et l'extension shield CAN-BUS:

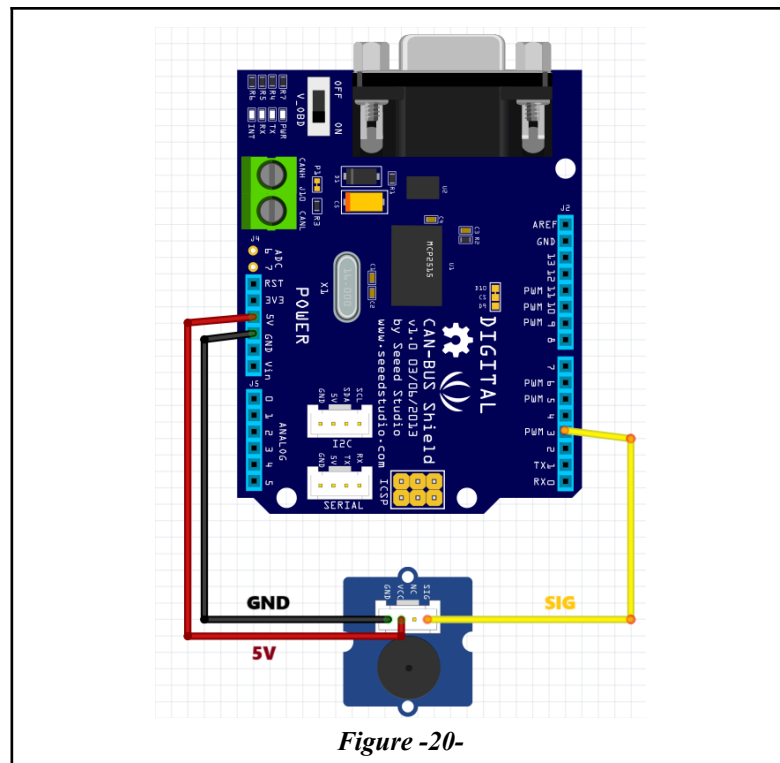


Figure -20-

Le *MCP2515* reçoit les instructions de l'Arduino (par SPI) pour envoyer ou écouter les trames **CAN**. Il supporte le protocole **CAN 2.0B**, et dans ce projet, il est configuré pour utiliser une horloge de 16 MHz (via un quartz sur le shield) et un débit **CAN** de 100 kbps.

Le *MCP2515* est interfacé avec l'Arduino à travers ces lignes SPI :

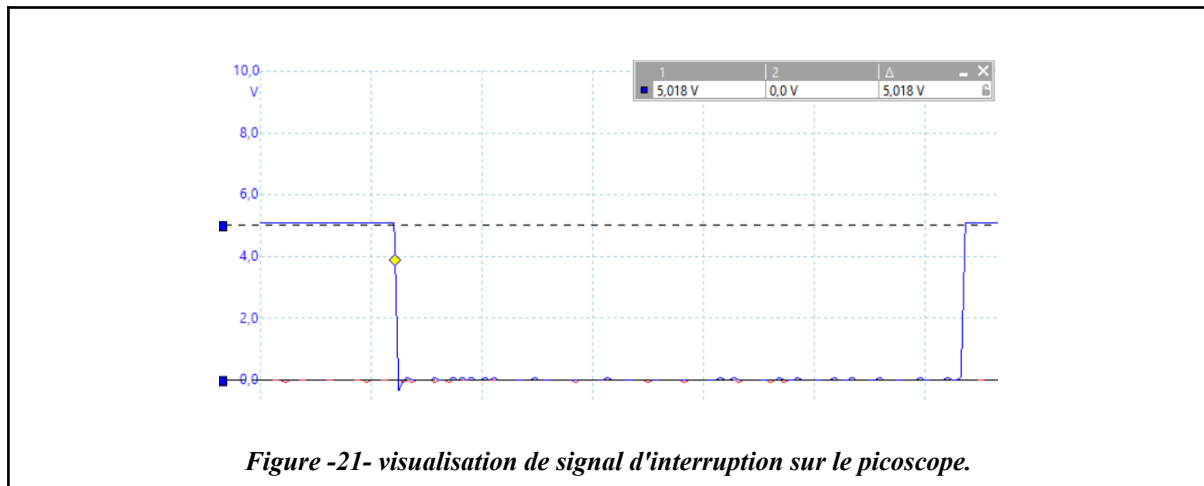
Broche Arduino	Fonction sur le shield CAN BUS v1.2	Rôle
D2	INT	Pour l'interruption en cas de message CAN entrant. (Le MCP2515 générant un interrupt sur INT , Arduino exécute <code>"mcp2515.readMessage()"</code> pour récupérer l'ACK ou les données.
D9	CS (Chip Select SPI)	Sélection du MCP2515 pendant la communication SPI (active le MCP2515 lors des échanges)
D11 (MOSI)	SPI Master-Out, Slave-In	Données envoyées par l'Arduino vers le MCP2515 (données de configuration ou de trames)
D12 (MISO)	SPI Master-In, Slave-Out	Données reçues du MCP2515 vers l'Arduino
D13 SCK (via header ICSP)	SPI Clock	Horloge SPI
GND	Masse	Référence électrique commune
5V	Alimentation du shield	Alimente le MCP2515 et le MCP2551

Le protocole SPI permet à l'Arduino (maître SPI) d'envoyer des commandes au MCP2515 (esclave SPI) pour écrire, lire ou envoyer des trames CAN.

C- Fonctionnement de la communication SPI (CS, MOSI, MISO, SCK) avec interruption:

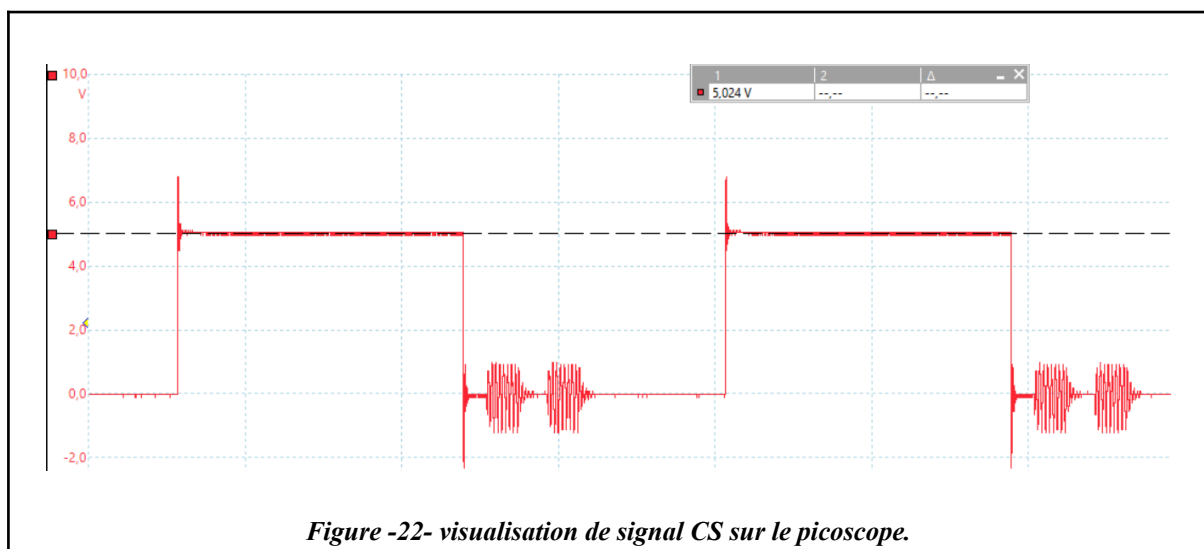
L'Arduino transmet les octets ci-dessus à travers le bus SPI vers le *MCP2515*, selon le protocole SPI standard :

1. Un message **CAN** est reçu par le contrôleur *MCP2515* depuis le bus **CAN**.
2. Le *MCP2515* génère une **interruption** matérielle en abaissant la broche **INT** (connectée à D2 de l'Arduino).

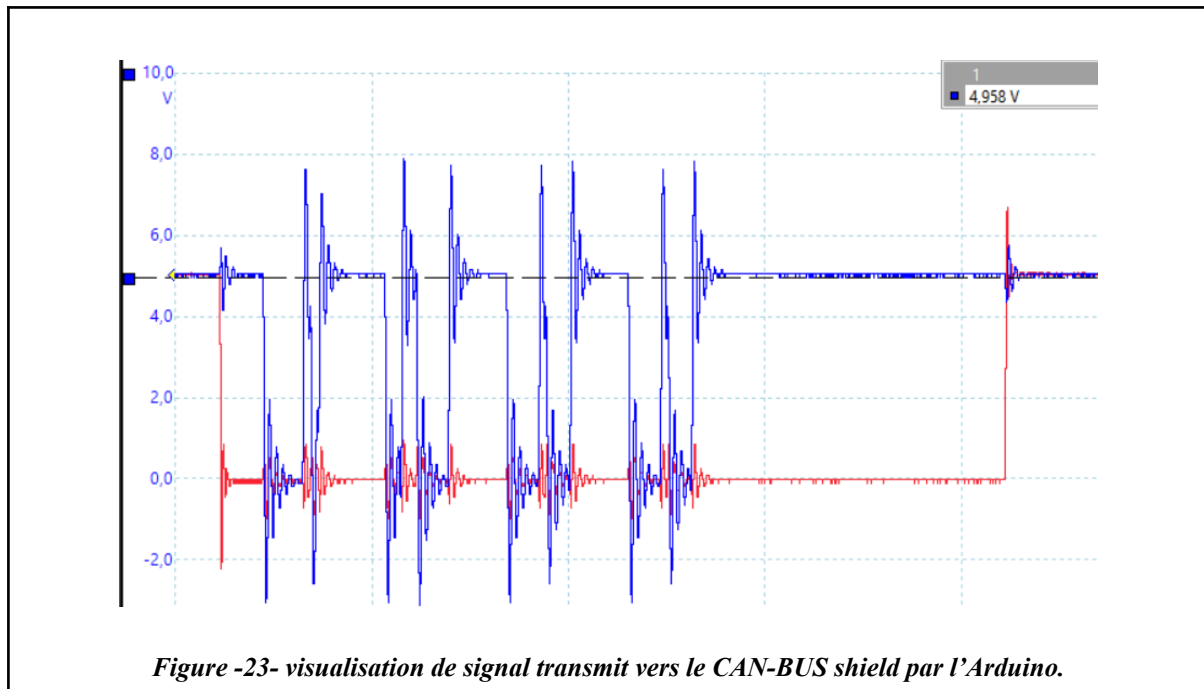


L'Arduino détecte cette interruption via INT0 et déclenche la fonction “**interruptionCAN()**”, qui met un flag “**messageRecu**” est mis à **true**

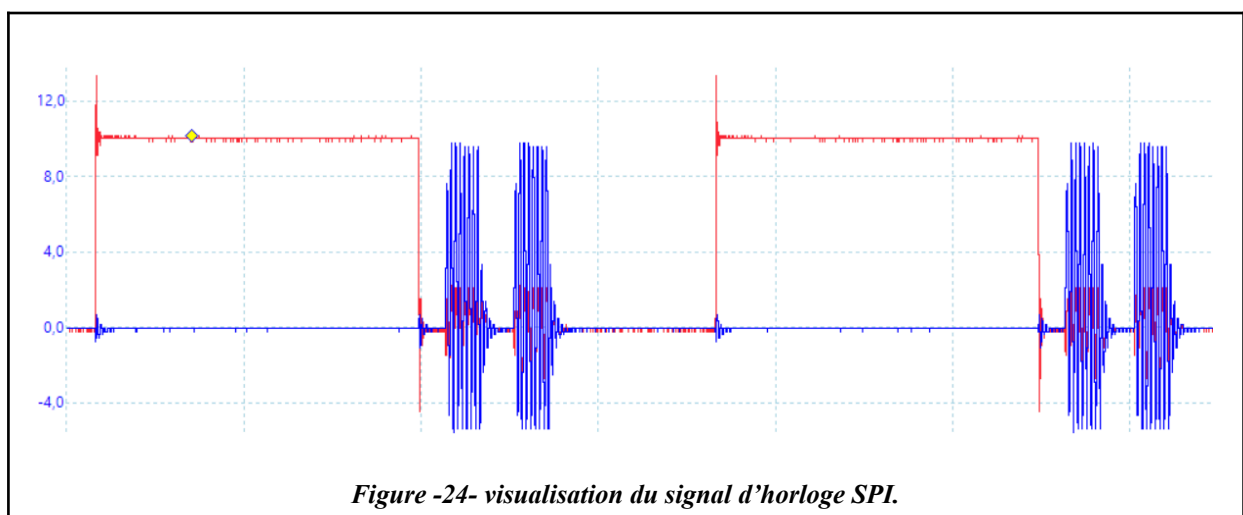
3. Le signal **CS (Chip Select)** est mis à LOW (**D9**) pour démarrer la communication SPI avec le *MCP2515*.



4. Les données sont transmises bit par bit de l'Arduino vers le *MCP2515* via la broche MOSI (D11).

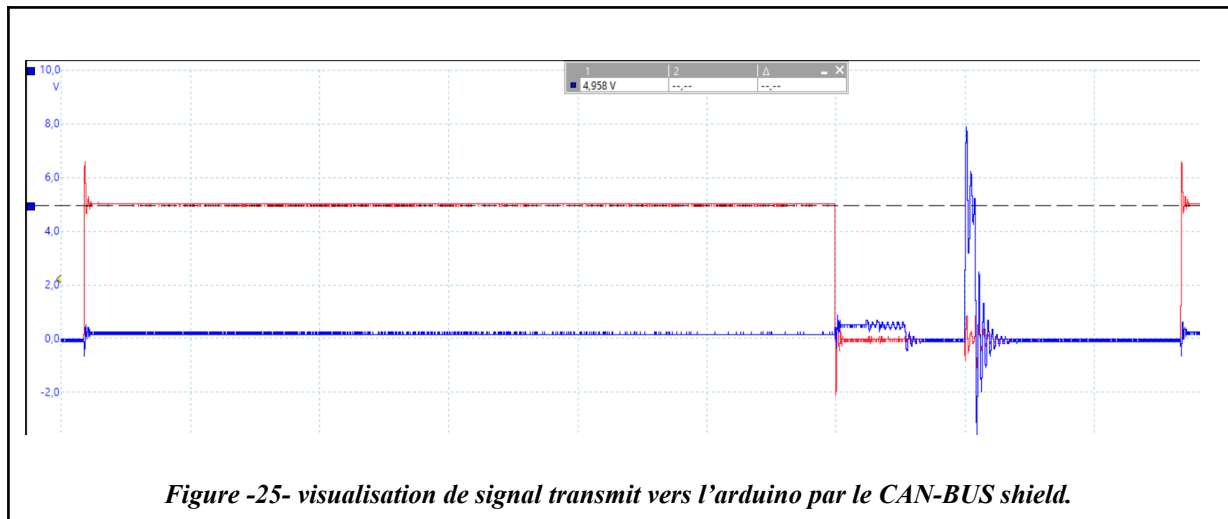


4. Le **signal d'horloge SCK (D13)** cadence chaque bit transmis



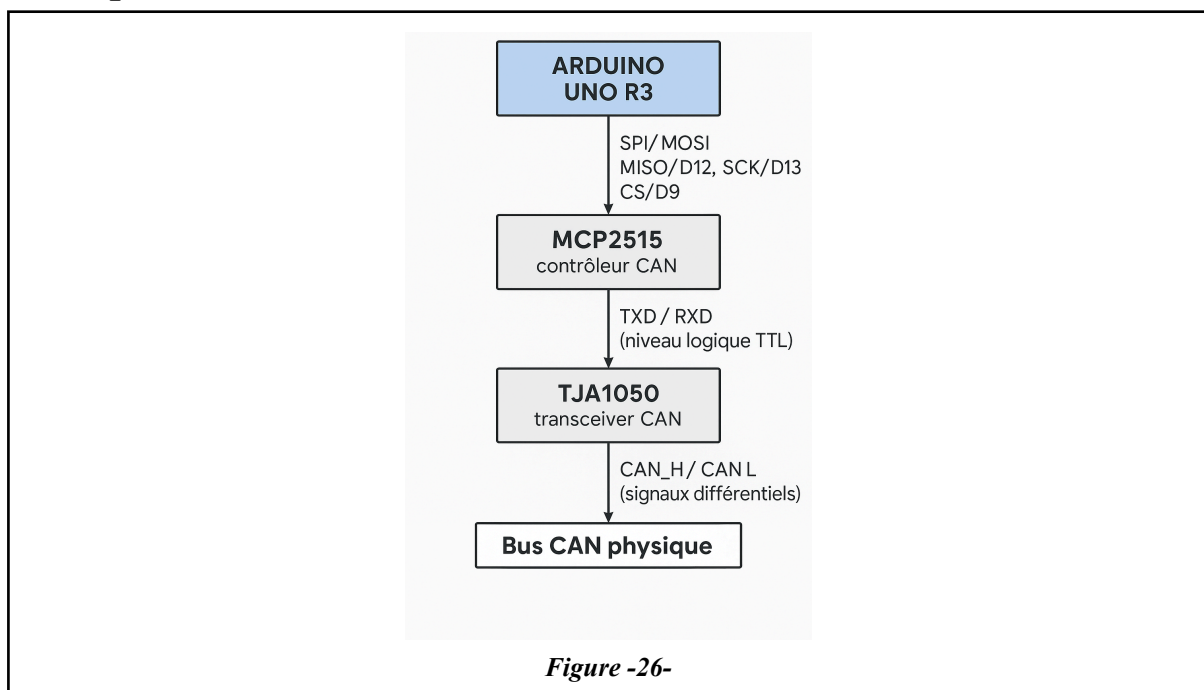
La fréquence de cette horloge est contrôlée par l'Arduino en fonction de la configuration SPI (de 1 MHz à 8 MHz).

5. Le *MCP2515* répond, s'il y a lieu, en renvoyant des données via la broche **MISO (D12)**.



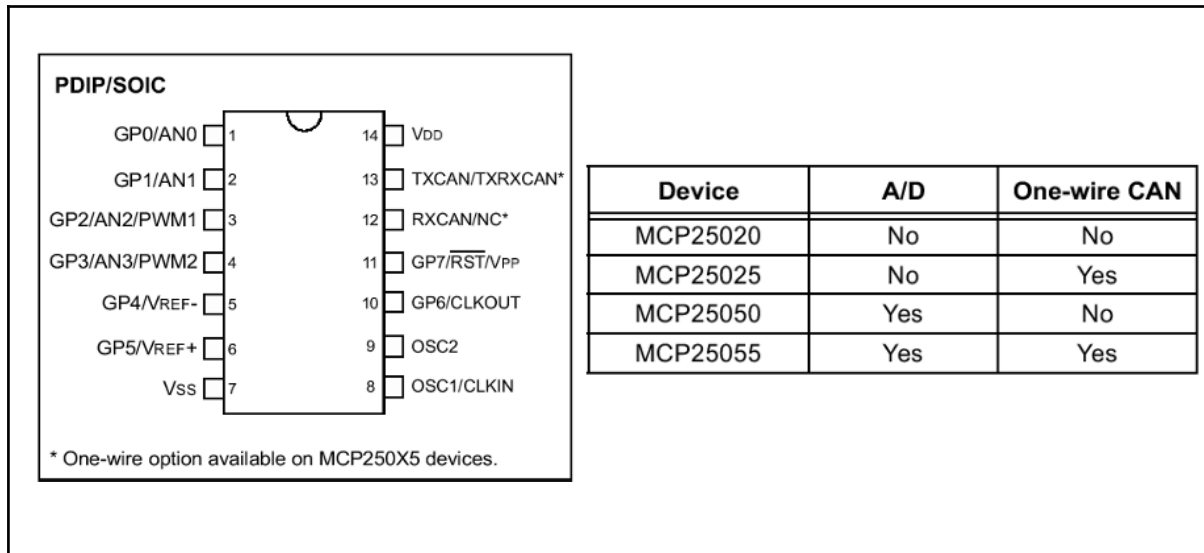
6. La communication SPI se termine, le **CS est remis à HIGH**. L'Arduino peut maintenant traiter les données reçues.

D- Séquence du flux SPI:



IV.5.2. MCP25050 (module esclave du CAN01A)

Le *MCP25050* est un composant développé par Microchip, combinant un contrôleur **CAN** conforme à la norme **CAN 2.0B** et une interface d'entrées/sorties numériques. Il est spécialement conçu pour fonctionner en tant que module esclave sur un bus **CAN**, sans nécessiter l'intégration d'un microcontrôleur externe. Ce composant est intégré au cœur des modules CAN01A du système *VMD* (comme les blocs feux ou le commodo), permettant leur interaction autonome sur le bus.



Le *MCP25050* est disponible en boîtier 14 broches (PDIP ou SOIC). Il dispose de 8 lignes I/O (GP0 à GP7) entièrement configurables individuellement en entrée ou sortie via des registres internes accessibles à distance via le **CAN**. À noter toutefois que la broche GP7 ne peut être utilisée en sortie. Deux broches (GP2 et GP3) peuvent être configurées en sortie PWM. Ces deux sorties peuvent être commandées indépendamment l'une de l'autre, fréquences et rapports cycliques sur 10 bits.

Enfin, côté interface **CAN**, le *MCP25050* permet des communications à des vitesses allant jusqu'à 1 Mbit/s, bien que dans le contexte du *VMD*. Certaines versions spéciales de ce composant autorisent également un mode de communication "*One-Wire*" (sur un seul fil), mais ce mode n'est pas utilisé dans le cadre de ce projet.

Sur le CAN01A :

- Les boutons du commodo sont lus via les entrées GP0–GP7, configurées en entrée.

GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
E	E	E	E	E	E	E	E

GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
klaxon	stop	Clignotant droit	Clignotant gauche	code	phare	warning	Veilleuse

E: Entrée TOR

- Les feux (Carte 4 sorties TOR) sont commandés via les sorties GP0–GP3.

GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
E	E	E	E	S	S	S	S

S: Sortie TOR

Feux Avant:

GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
Etat clignotant	Etat phare	Etat code	Etat veilleuse	clignotant	phare	code	Veilleuse

Feux arrières:

GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
Etat GP3	Etat clignotant	Etat code	Etat veilleuse	(klaxon)	clignotant	code	Veilleuse

La commande klaxon est active sur le module feux arrière gauche.

- Le module envoie périodiquement une trame OM indiquant l'état logique de ses broches, facilitant la lecture côté Arduino.

Ce composant simplifie considérablement l'implémentation côté matériel dans les modules du *VMD*, en masquant la complexité du CAN.

IV.5.3. Horloge et vitesse de communication

Deux constantes sont définies dans le code Arduino pour configurer le contrôleur CAN :

```
"#define CAN_SPEED CAN_100KBPS"
```

```
"#define CAN_CLOCK MCP_16MHZ"
```

- **CAN_SPEED** : indique la vitesse souhaitée de communication CAN, ici 100 kilobits par seconde.
- **CAN_CLOCK** : précise la fréquence du quartz utilisé par le MCP2515 (16 MHz dans le shield Sseed Studio v1.1).

La cohérence entre ces deux paramètres est essentielle : le *MCP2515* utilise des diviseurs d'horloge pour générer la synchronisation du protocole CAN. Si la fréquence d'horloge ou la vitesse CAN sont mal configurées, cela provoque des erreurs de communication, comme des ACK manquants ou des messages non lus par les autres nœuds.

Le choix de 100 kbps avec une horloge de 16 MHz correspond à une configuration standard et stable, parfaitement adaptée à la topologie du système *VMD* (peu de nœuds, faible trafic). Cela permet aussi une bonne visualisation des trames via le PicoScope et évite les erreurs dues à des trames trop rapides ou mal synchronisées.

V. Implémentation logicielle sur Arduino

V.1. Organisation générale du code Arduino

L'architecture logicielle repose sur une approche modulaire et réutilisable, intégrant plusieurs bibliothèques essentielles. La bibliothèque **"SPI.h"** est utilisée pour la communication série entre l'Arduino Uno R3 et le contrôleur *MCP2515*, via le protocole SPI. La bibliothèque **"mcp2515.h"** permet quant à elle de configurer et d'utiliser le bus CAN à l'aide de fonctions de haut niveau.

Chaque TP commence par l'initialisation du module *MCP2515* : réinitialisation, configuration du débit (ici CAN_100KBPS) et passage en mode normal. Cette initialisation est encapsulée dans des fonctions comme **"initialiserCAN()"** ou **"initCAN()"**.

La configuration des modules *VMD* (feux ou commodo) se fait via l'envoi de trames CAN de type IM (*Input Message*), notamment pour les registres GPDDR (0x1F) qui détermine la direction des broches (entrée/sortie) et GPLAT (0x1E) qui définit leur état logique. Par exemple, la ligne suivante configure les 4 bits de poids faibles du registre GPDDR en sortie pour le bloc optique :

```
frame.data[0] = REG_GPDDR;
```

```
frame.data[1] = 0x0F; // Masque
```

```
frame.data[2] = 0xF0; // Bits en sortie (valeur)
```

Les structures de données (par exemple *EtatFeu* ou *EtatCommodo*) permettent de gérer proprement l'état des feux ou boutons, et facilitent la maintenance du code. Une logique de temporisation basée sur *millis()* est utilisée pour déclencher des changements d'état de manière non-bloquante (TP1, TP3, TP4), tandis que les trames CAN sont envoyées selon des séquences cycliques ou conditionnées à des entrées utilisateur.

V.2. Fonctionnement de l'interruption matérielle

Le système repose largement sur l'utilisation d'interruptions matérielles via la broche INT du *MCP2515*, reliée à l'entrée D2 de l'Arduino. Cette interruption est déclenchée chaque fois qu'un message CAN est reçu.

Dans chaque TP, la fonction *attachInterrupt()* est utilisée pour lier l'interruption externe à une ISR (Interrupt Service Routine) spécifique :

```
attachInterrupt(digitalPinToInterrupt(INT_PIN), interruptionCAN, FALLING);
```

L'ISR *interruptionCAN()* se contente de définir un flag booléen (*messageRecu* ou *ackRecu*), ce qui permet de différer le traitement dans la boucle principale (*loop()*) pour ne pas alourdir l'exécution de l'interruption elle-même.

L'avantage de ce mécanisme est qu'il supprime le besoin de "polling" actif (interrogation continue du bus). Cela améliore la réactivité (détection immédiate de trames entrantes) et optimise l'efficacité énergétique du système, particulièrement utile dans les architectures temps réel ou à ressources limitées. Dans le TP3, cette méthode est essentielle pour assurer la réception d'un acquittement AIM sans bloquer le système.

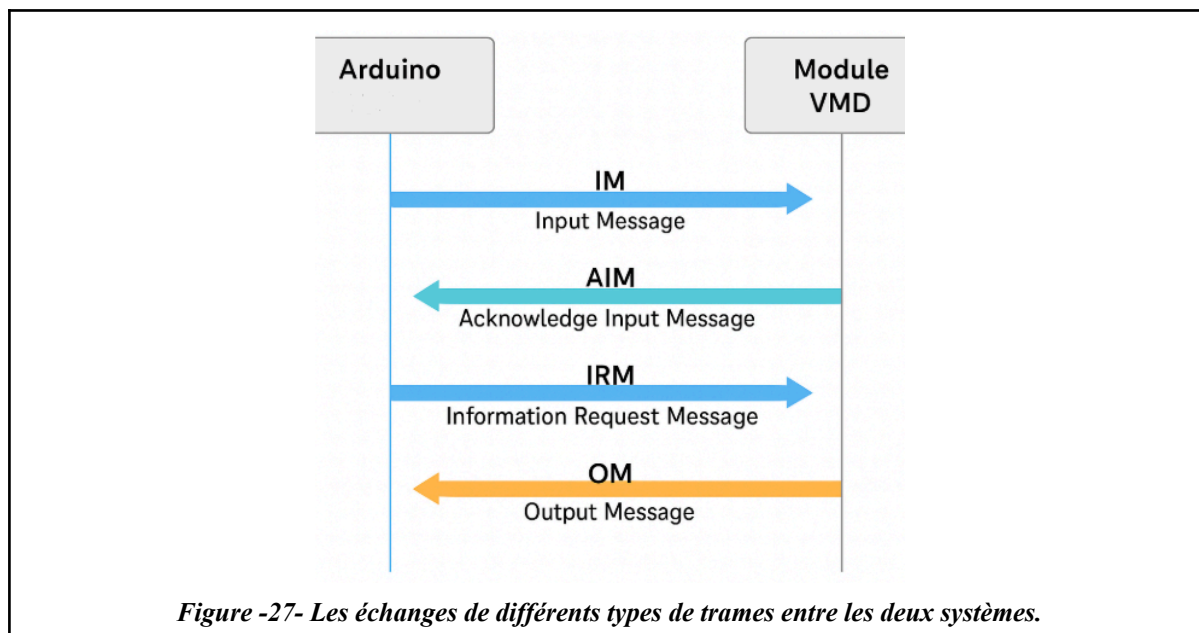
V.3. Décodage et envoi des trames

Plusieurs types de trames CAN sont utilisés dans les TPs :

- **IM (Input Message)** : trames de commande envoyées par l'Arduino vers les modules VMD. Elles contiennent un registre (ex : GPDDR, GPLAT), un masque, et une valeur.
- **OM (Output Message)** : trames automatiques générées par les modules (notamment le commodo), utilisées pour signaler l'état des boutons ou capteurs (TP2, TP4).
- **IRM (Information Request Message)** : trames de requête envoyées pour demander explicitement l'état d'un module. Le TP4 l'utilise périodiquement pour obtenir l'état

du commodo.

- **AIM (*Acknowledgement IM*)** : trames d'acquittement envoyées par les modules pour signaler la réception et l'exécution d'une commande. Leur réception est surveillée par interruption dans les TP1, TP3, et TP2.



L'envoi de trames utilise une structure **can_frame**, avec un identifiant étendu (can_id | CAN_EFF_FLAG), une longueur (can_dlc = 3), et trois octets de données :

Champ	Valeur
can_id	0x0E880000 (module FAD) avec CAN_EFF_FLAG
can_dlc	3 (3 octets de données)
trame.data[0]	registre // ex: 0x1E (GPLAT)
trame.data[1]	masque // bits concernés (ex: 0x0F)
trame.data[2]	valeur // valeur binaire à appliquer

Le masque binaire permet de cibler des bits spécifiques du registre sans modifier les autres, ce qui est essentiel pour les opérations de type logique TOR (tout ou rien).

Les TP intègrent aussi des mécanismes de temporisation logicielle (TP3, TP4) pour contrôler les clignotements (800 ms à 1.6 s), ou limiter la fréquence d'envoi des trames (anti-spam CAN avec ***CAN_SEND_DELAY***).

VI. Réalisation des travaux pratiques avec Arduino

L'objectif global des travaux pratiques est de remplacer la carte *EID210* par une Arduino Uno R3 équipée d'un shield CAN-BUS v1.1 (*MCP2515*), afin de piloter le système de signalisation du *VMD* (module CAN01A).

Les exercices TP1 à TP4 reproduisent fidèlement les scénarios originaux, en exploitant le protocole CAN pour commander les feux avant/arrière et lire l'état du commodo.

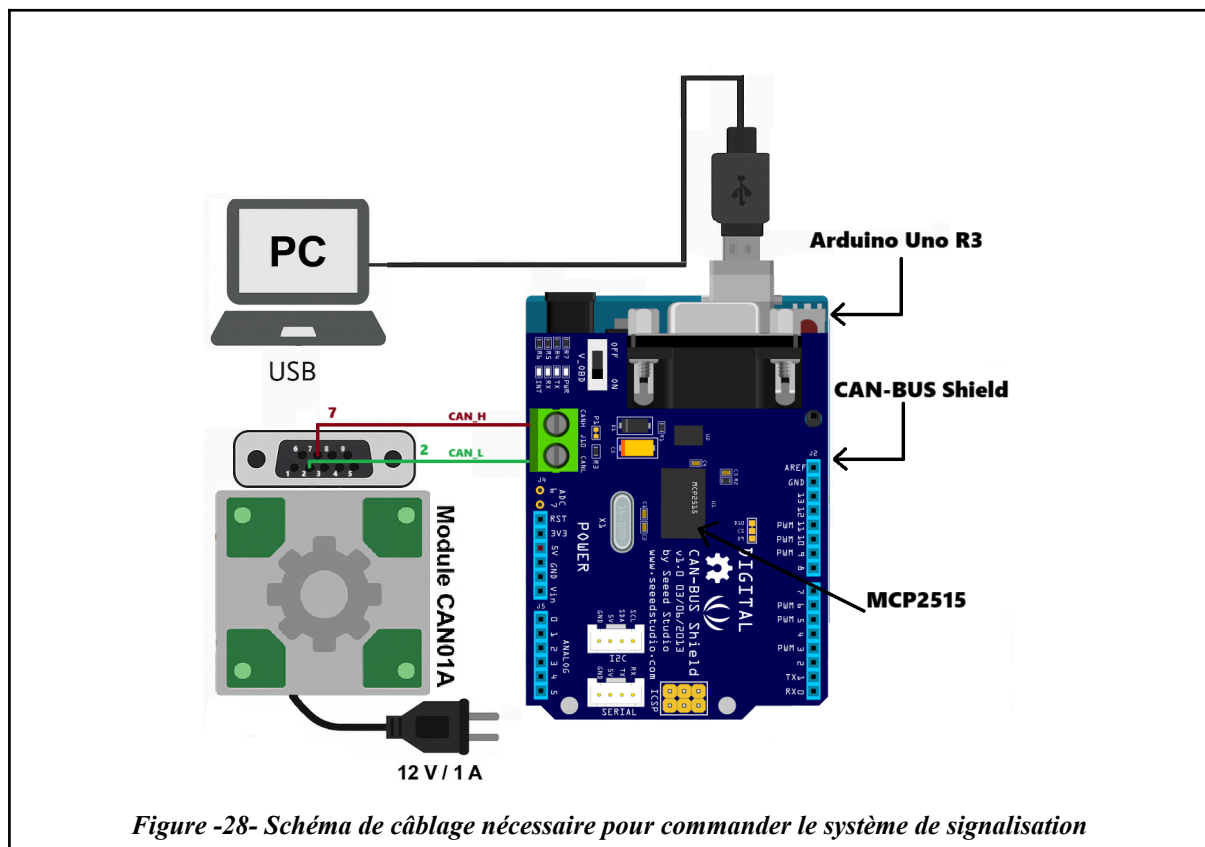


Figure -28- Schéma de câblage nécessaire pour commander le système de signalisation

VI.1. TP1 : Faire commuter les lampes d'un bloc optique

Objectif

Le TP1 vise à mettre en œuvre un chenillard lumineux sur un bloc optique spécifique du *VMD* afin de vérifier la maîtrise de l'envoi de trames **CAN** depuis l'Arduino. Il s'agit d'un premier exercice d'écriture sur le bus **CAN**, où l'Arduino contrôle séquentiellement l'allumage de plusieurs sorties du *MCP25050* (via registre GPLAT), avec un temps défini entre chaque étape.

L'objectif pédagogique est de maîtriser l'envoi de trames IM au module *MCP25050* pour commander les sorties **GPx** associées aux feux avant et arrière, en respectant un ordre séquentiel et un délai constant entre chaque étape.

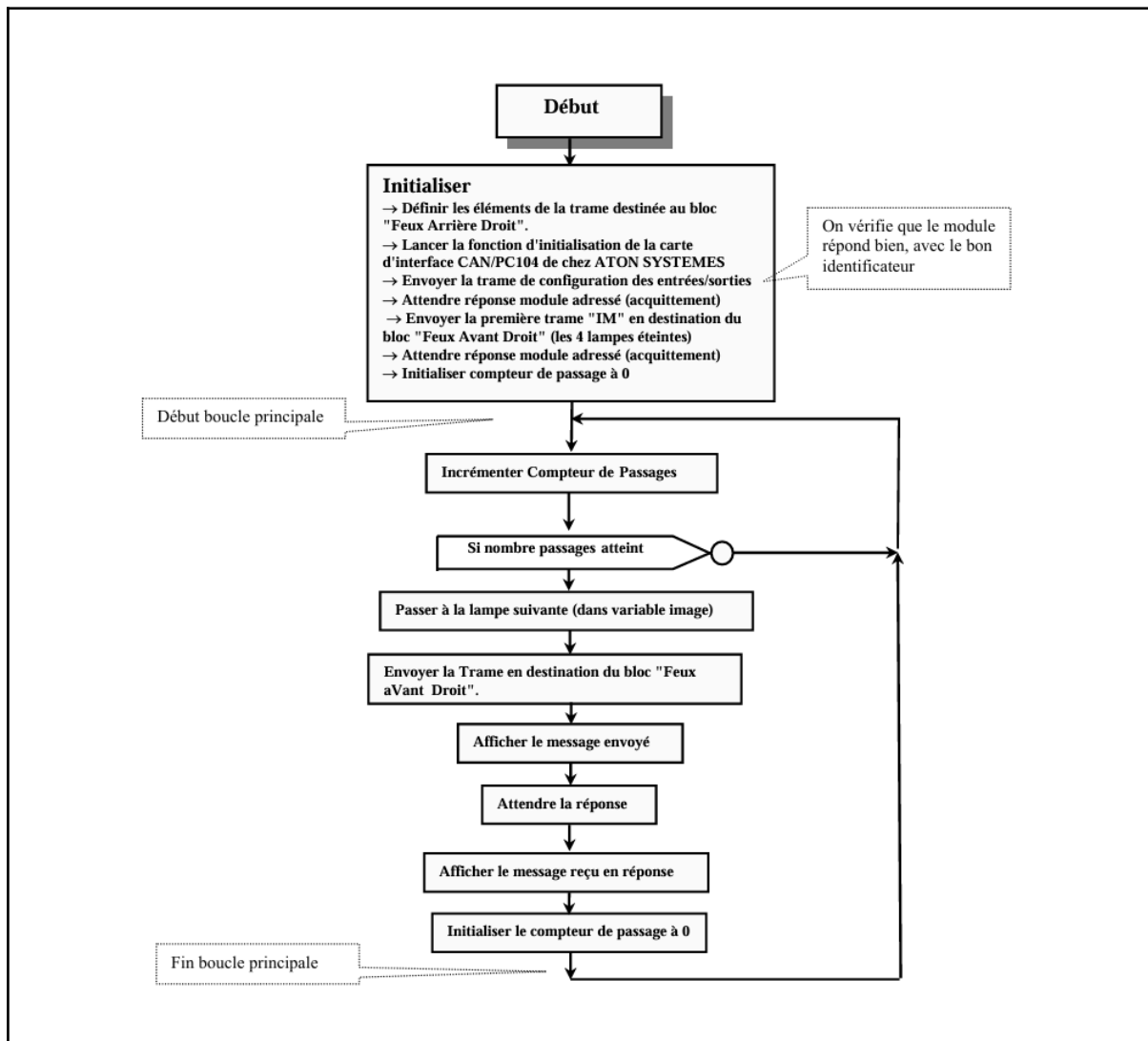
Ce TP permet de se familiariser avec :

- L'adressage des modules via identifiant **CAN** étendu (29 bits).
- La construction de trames IM.
- La gestion des masques et valeurs d'écriture sur le registre GPLAT.

Cahier des charges:

- A intervalles de temps réguliers, on interroge le module sur lequel est connecté le commodo lumières afin de connaître son état.
- Les trames reçues ou envoyées sur le bus **CAN** sont affichées.
- La temporisation est de type "logiciel" (comptage du nombre de passages dans la boucle principale)
- Les différentes commandes imposées par la position de la manette commodo seront affichées individuellement.

Organigramme:



Fonctionnement:

Le TP1 a pour objectif de créer un système permettant de contrôler séquentiellement les feux d'un véhicule en utilisant le protocole **CAN**, largement répandu dans l'industrie automobile. Le choix de l'Arduino couplé au shield *MCP2515* s'est imposé pour sa simplicité de mise en œuvre tout en offrant une base solide pour comprendre les principes fondamentaux des communications **CAN**.

La configuration matérielle initiale est cruciale : la broche CS (Chip Select) du *MCP2515* est connectée à la broche 9 de l'Arduino, tandis qu'une broche d'interruption (*INT_PIN = 2*) est réservée pour détecter les trames envoyées vers l'arduino.

```

16 // ----- CONFIGURATION MATÉRIELLE -----
17
18 const int SPI_CS_PIN = 9;    // Broche CS pour MCP2515
19 const int INT_PIN      = 2;  // Broche d'interruption (INT) du MCP2515
20
21 MCP2515 mcp2515(SPI_CS_PIN); // Instance du contrôleur CAN
22 volatile bool messageRecu = false; // Flag mis à jour par interruption

```

Cette interruption matérielle est essentielle car elle permet une réaction immédiate aux réponses des modules sans avoir recours à un polling actif qui consommerait inutilement des ressources processeur.

La structure **ModuleConfig** joue un rôle central dans ce programme en encapsulant toutes les informations nécessaires à la communication avec chaque module de feux : un nom lisible pour le débogage, un identifiant CAN pour les commandes (**ID_IM**), un identifiant pour les acquittements (**ID_AIM**), et une information sur la position avant/arrière.

```

24 // ----- SÉLECTION DU MODULE À CONTRÔLER -----
25
26 // Décommenter **UN SEUL** module :
27 #define MODULE_AVANT_DROIT
28 // #define MODULE_AVANT_GAUCHE
29 // #define MODULE_ARRIERE_DROIT
30 // #define MODULE_ARRIERE_GAUCHE
31
32 struct ModuleConfig {
33     const char* nom;        // Nom lisible du module
34     uint32_t id_im;         // ID Input Message (commande)
35     uint32_t id_aim;        // ID Acquittement du module
36     bool is_avant;          // true = avant / false = arrière
37 };
38
39 const ModuleConfig modules[] = {
40     {"Feux avant droit", 0xE80000, 0xEA0000, true},
41     {"Feux avant gauche", 0xE08000, 0xE20000, true},
42     {"Feux arrière droit", 0xF88000, 0xFA0000, false},
43     {"Feux arrière gauche", 0xF08000, 0xF20000, false}
44 };
45
46 #if defined(MODULE_AVANT_DROIT)
47     #define MODULE_INDEX 0
48 #elif defined(MODULE_AVANT_GAUCHE)
49     #define MODULE_INDEX 1
50 #elif defined(MODULE_ARRIERE_DROIT)
51     #define MODULE_INDEX 2
52 #elif defined(MODULE_ARRIERE_GAUCHE)
53     #define MODULE_INDEX 3
54 #else
55     #error "Aucun module sélectionné!"
56 #endif
57
58 const ModuleConfig& module = modules[MODULE_INDEX];

```

Cette organisation en structure permet une maintenance aisée et une grande flexibilité lorsqu'il s'agit d'ajouter ou de modifier des modules.

La séquence d'éclairage est définie dans des tableaux d'états (*sequenceAvant*, *chenillardArriereGauche*, etc.) où chaque entrée associe une valeur hexadécimale à une description textuelle. Cette valeur correspond en réalité au motif binaire qui sera envoyé au registre GPLAT du circuit *MCP25050* pour activer les sorties spécifiques. Par exemple, la valeur **0x01** (00000001 en binaire) active la broche GP0, typiquement utilisée pour la veilleuse.

GP3	GP2	GP1	GP0
clignotant	phare	code	Veilleuse

```

67 // ----- SÉQUENCES DES ÉTATS DES FEUX -----
68
69 struct EtatFeu {
70     uint8_t valeur;
71     const char* description;
72 };
73
74 // Feux avant
75 const EtatFeu sequenceAvant[] = {
76     {0x01, "Veilleuse (GP0)"},
77     {0x02, "Code (GP1)"},
78     {0x04, "Phare (GP2)"},
79     {0x08, "Clignotant (GP3)"},
80     {0x00, "Tous éteints"}
81 };
82
83 // Feux arrière gauche
84 const EtatFeu chenillardArriereGauche[] = {
85     {0x08, "Veilleuse (GP0)"},
86     {0x04, "Clignotant (GP1)"},
87     {0x02, "Stop (GP2)"},
88     {0x01, "Klaxon (GP3)"},
89     {0x00, "Tous éteints"}
90 };
91
92 // Feux arrière droit
93 const EtatFeu chenillardArriereDroit[] = {
94     {0x08, "Veilleuse (GP0)"},
95     {0x04, "Clignotant (GP1)"},
96     {0x02, "Stop (GP2)"},
97     {0x01, "Libre (GP3)"},
98     {0x00, "Tous éteints"}
99 };

```

La boucle principale utilise `millis()` pour gérer les temporisations, une méthode non bloquante bien préférable à `delay()` car elle permet au microcontrôleur de continuer à traiter d'autres tâches pendant l'attente.

```
218 void loop() {  
219     // Gestion cyclique des feux  
220     if (millis() - dernierChangement >= INTERVALLE_CHANGEMENT) {  
221         dernierChangement = millis();  
    }
```

Lorsque le temps est écoulé, la fonction `envoyerCommande()` est appelée pour construire et envoyer la trame CAN appropriée. Cette fonction prépare soigneusement chaque élément de la trame :

- L'identifiant étendu (`module.id_im | CAN_EFF_FLAG`)
- La longueur des données (3 octets)

et les trois paramètres critiques:

- L'adresse du registre (GPLAT à 0x1E)
- Le masque (0x0F pour ne modifier que les 4 bits de poids faible)
- La valeur à écrire.

L'utilisation d'un masque est particulièrement importante car elle permet de modifier uniquement les bits concernés par la commande courante sans affecter les autres sorties.

```
133 // ===== ENVOI COMMANDE CAN (IM) =====  
134  
135 bool envoyerCommande(uint8_t registre, uint8_t masque, uint8_t valeur) {  
136     struct can_frame trame;  
137  
138     trame.can_id = module.id_im | CAN_EFF_FLAG;  
139     trame.can_dlc = 3;  
140     trame.data[0] = registre;  
141     trame.data[1] = masque;  
142     trame.data[2] = valeur;  
    }
```

L'interruption est déclenchée sur front descendant (FALLING) lorsque le contrôleur CAN reçoit une trame.

```
190 pinMode(INT_PIN, INPUT);
191 attachInterrupt(digitalPinToInterrupt(INT_PIN), interruptionCAN, FALLING);
```

Lorsqu'une trame CAN est détectée, la routine d'interruption `interruptionCAN()` est exécutée elle se contente de positionner un simple drapeau booléen volatile `messageRecu` à true.

Suivant ainsi le principe de la routine d'interruption la plus courte possible. Ce drapeau est ensuite testé dans la boucle principale (`loop()`), où la véritable gestion des messages reçus a lieu - une séparation des responsabilités essentielle pour maintenir la réactivité du système. Lors du traitement, la fonction lit toutes les trames en attente via `mcp2515.readMessage()` et filtre spécifiquement les trames AIM (Acquittement Input Message) en comparant leur identifiant (`id_recue`) avec celui attendu (`module.id_aim`). Ce mécanisme permet de vérifier que chaque commande envoyée aux feux a bien été reçue et exécutée par le module cible avant de passer à l'état suivant.

```
234 // Traitement des messages reçus (interruption)
235 if (messageRecu) {
236     messageRecu = false;
237
238     struct can_frame trame;
239     while (mcp2515.readMessage(&trame) == MCP2515::ERROR_OK) {
240         uint32_t id_recue = trame.can_id & 0x1FFFFFFF;
241
242         if (id_recue == module.id_aim) {
243             Serial.print("[RECEPTION] ✓ AIM reçu - ID: 0x");
244             Serial.println(module.id_aim, HEX);
245         } else {
246             Serial.print("[RECEPTION] ✗ Trame inconnue - ID: 0x");
247             Serial.println(id_recue, HEX);
248         }
249     }
250 }
251 }
```

Résultats obtenus

Le chenillard est fluide et stable, sans perte de trame, et les temps d'allumage sont respectés.

La visualisation et l'analyse des trames CAN avec le *PicoScope 3204D MSO* s'effectue selon une méthodologie rigoureuse :

1. Configuration matérielle initiale :

- Connecter CAN_H sur le canal A et CAN_L sur le canal B
- Utiliser des sondes différentielles pour une meilleure immunité au bruit

- Vérifier la mise à la terre commune pour éviter les offsets

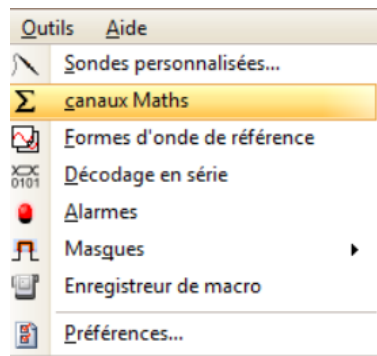
2. Paramétrage logiciel (PicoScope 6) :

a) Configuration des entrées :

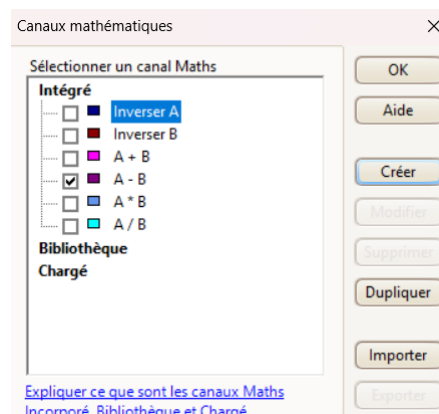
- Régler les échelles verticales à $\pm 5V$ (typique pour CAN)
- Ajuster la base de temps à (*****) pour visualiser une trame complète

b) Création du signal différentiel :

- Menu Outils > Canaux maths

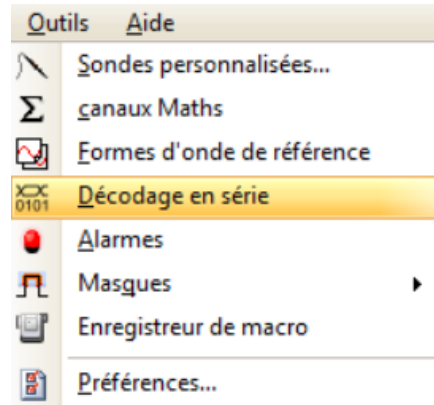


- Sélectionner l'opération A-B pour obtenir le signal différentiel CAN (Cette opération élimine le bruit commun et isole le signal utile)



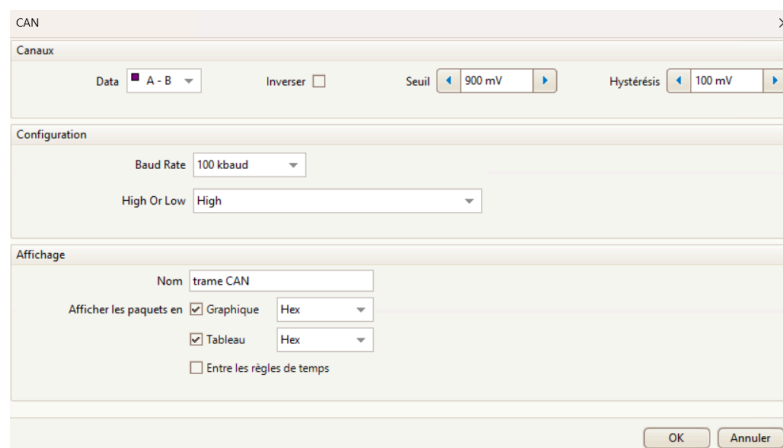
3. Configuration du décodeur CAN :

- Onglet Outils > Décodeur série > **CAN**

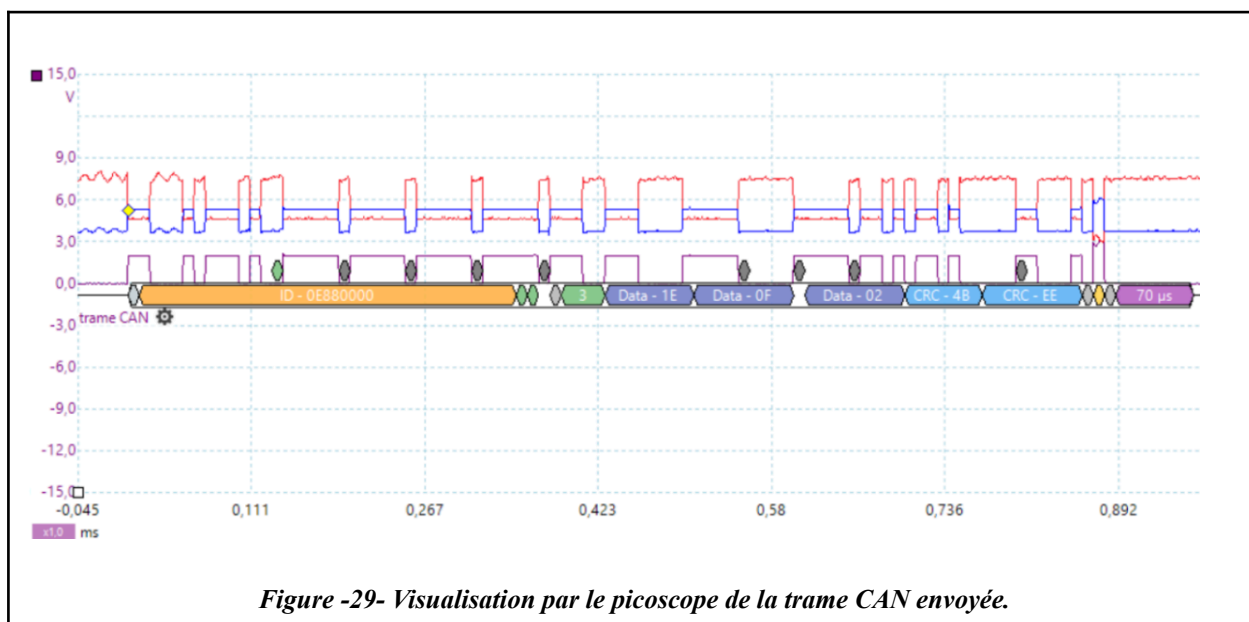


Paramètres critiques :

- Source : "Maths (A-B)" (signal différentiel créé)
- Baud rate : 100 kbit/s (standard automobile)
- Niveau : High (logique **CAN** dominante/récessive)
- Format d'affichage : HEX (plus lisible pour l'analyse protocolaire)



On visualise la trame IM envoyée au module vers le bloc avant droit pour allumer la led du feu “code” en envoyant dans le champ de donnée “Data - 02” = 0010 en binaire (cad 1 en GP1).



Lorsque la commande est transmise au module CAN01A sous forme de trame IM, le composant *MCP25050*, qui gère les entrées/sorties du module, exécute l’action demandée puis génère automatiquement un message de retour sur le bus *CAN*. Ce message, identifiable par un identifiant spécifique (0x0EA00000 dans le cas observé), joue le rôle d’accusé de réception interne ou de trame système. Sa fonction principale n’est pas de transporter des données applicatives, mais simplement de signaler au maître que la commande a bien été reçue et traitée. Pour cette raison, son champ DLC (*Data Length Code*) est égal à zéro, ce qui indique qu’aucun octet de données n’est inclus dans la trame. Toutefois, comme toute trame **CAN** valide, elle conserve les champs obligatoires, notamment le CRC (*Cyclic Redundancy Check*) pour vérifier l’intégrité et le ACK (*Acknowledgement*) qui confirme la bonne réception par au moins un nœud du réseau. Ce mécanisme, assimilable à un AIM simplifié, garantit la fiabilité de la communication en assurant au contrôleur principal qu’une action déclenchée par une trame IM a bien été prise en compte par le module esclave.

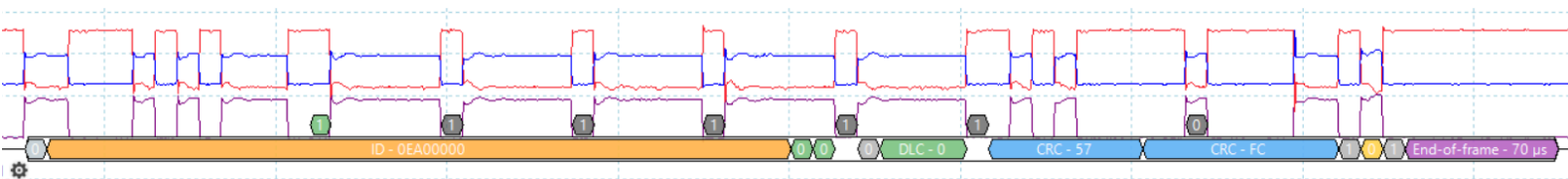


Figure -30- Trame de retour depuis le MCP25050

Visualisation de la séquence du fonctionnement chenillard sur le serial monitor d'ArduinoIDE:

```

⚠ Changement d'état : Tous éteints
[ENVOI] [RXF1] ID: 0xE880000 | Registre: 0x1E | Masque: 0xF | Valeur: 0x0
[RECEPTION] ✅ AIM reçu - ID: 0xEA00000

⚠ Changement d'état : Veilleuse (GP0)
[ENVOI] [RXF1] ID: 0xE880000 | Registre: 0x1E | Masque: 0xF | Valeur: 0x1
[RECEPTION] ✅ AIM reçu - ID: 0xEA00000

⚠ Changement d'état : Code (GP1)
[ENVOI] [RXF1] ID: 0xE880000 | Registre: 0x1E | Masque: 0xF | Valeur: 0x2
[RECEPTION] ✅ AIM reçu - ID: 0xEA00000

⚠ Changement d'état : Phare (GP2)
[ENVOI] [RXF1] ID: 0xE880000 | Registre: 0x1E | Masque: 0xF | Valeur: 0x4
[RECEPTION] ✅ AIM reçu - ID: 0xEA00000

⚠ Changement d'état : Clignotant (GP3)
[ENVOI] [RXF1] ID: 0xE880000 | Registre: 0x1E | Masque: 0xF | Valeur: 0x8
[RECEPTION] ✅ AIM reçu - ID: 0xEA00000

⚠ Changement d'état : Tous éteints
[ENVOI] [RXF1] ID: 0xE880000 | Registre: 0x1E | Masque: 0xF | Valeur: 0x0
[RECEPTION] ✅ AIM reçu - ID: 0xEA00000

```

VI.2. TP2 : Acquérir l'état du commodo feux

Objectif

Ce TP vise à lire l'état des boutons du commodo (module CAN01A) via des trames OM et à commander immédiatement les feux correspondants. Le but pédagogique est d'apprendre à gérer la réception de trames **CAN**, le décodage des états d'entrées et l'envoi de commandes en réaction, avec une latence minimale.

Ce TP introduit :

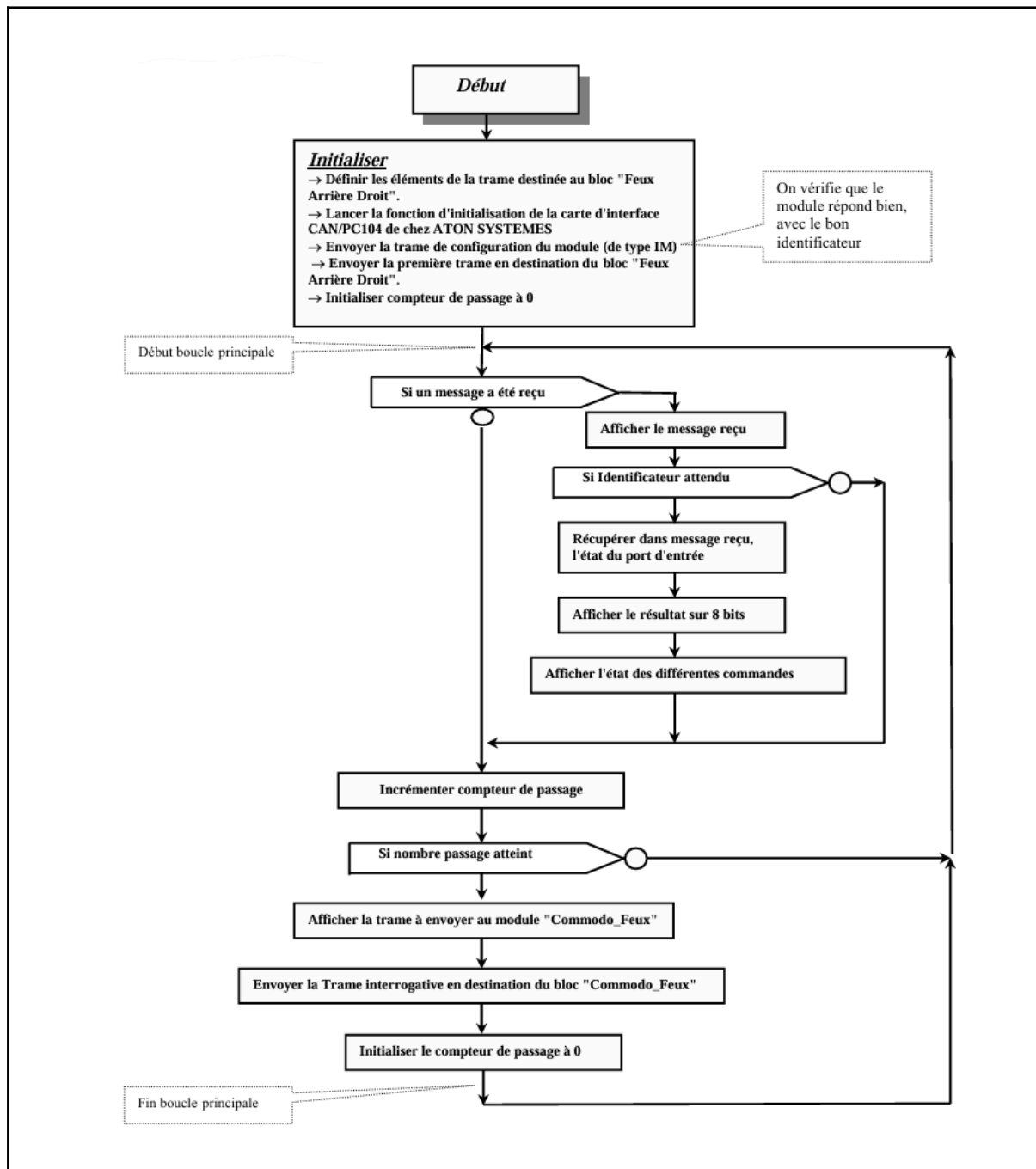
- La lecture sur le bus **CAN** (réception de trames).
- L'utilisation d'interruptions matérielles pour déclencher la lecture des trames.
- La correspondance entre bits d'entrée et action sur les modules de sortie. Il permet également d'intégrer des périphériques additionnels (ex. buzzer pour le klaxon) et de vérifier la réactivité de la communication.

Cahier des charges:

A intervalles de temps réguliers, on interroge le module sur lequel est connecté le commodo lumières afin de connaître son état.

- Les trames reçues ou envoyées sur le bus **CAN** sont affichées.
- La temporisation est de type "logiciel" (comptage du nombre de passages dans la boucle principale)
- Les différentes commandes imposées par la position de la manette commodo seront affichées individuellement.

Organigramme:



Fonctionnement:

Le TP2 constitue une évolution majeure par rapport au TP1 en introduisant une interface de contrôle complète permettant d'interpréter les actions d'un conducteur via un commodo (volant de commande) et de les traduire en commandes CAN. Le système repose sur une architecture matérielle composée d'une carte Arduino Uno R3 équipée d'un shield CAN intégrant le contrôleur *MCP2515* et le transceiver *MCP2551*, avec une broche d'interruption (INT) connectée à la broche D2 de l'Arduino pour une gestion optimale des événements CAN.

La configuration initiale du système implique une initialisation rigoureuse du bus CAN à 100 kbps (conformément à la norme ISO 11898 pour les applications automobiles) avec une horloge à 16 MHz

```
11 //===== CONFIGURATION MATÉRIELLE / CAN =====
12
13 const int SPI_CS_PIN = 9;
14 const int INT_PIN = 2;
15 const int BUZZER_PIN = 3; // Buzzer connecté sur D3
16
17 MCP2515 mcp2515(SPI_CS_PIN);
18
19 #define CAN_SPEED CAN_100KBPS
20 #define CAN_CLOCK MCP_16MHZ
```

suivie de la configuration des registres du *MCP25050*. Plus précisément, le registre GPDDR (General Purpose Data Direction Register - adresse 0x1F) est positionné à 0xFF pour configurer toutes les broches GP0 à GP7 en entrées, permettant ainsi la lecture des états des boutons du commodo.

```
61 void configurerModuleCommodoEnEntree() {
62     struct can_frame trame;
63     trame.can_id = ID_IM_COMMODO | CAN_EFF_FLAG;
64     trame.can_dlc = 3;
65     trame.data[0] = REG_GPDDR;
66     trame.data[1] = 0xFF;
67     trame.data[2] = 0xFF;
68     mcp2515.sendMessage(&trame);
69 }
```

La réception des données s'effectue via une interruption matérielle déclenchée sur front descendant de la broche INT, assurant une réactivité optimale du système. Lorsqu'une trame OM (*Output Message* - identifiant **0x05400000**) est reçue, le programme procède à l'extraction de l'état des entrées depuis le second octet de données (**data[1]**), auquel il applique une inversion logique nécessaire car le *MCP25050* utilise une logique active-bas (0V correspond à un bouton pressé). L'état ainsi obtenu est comparé à la valeur précédente stockée dans **dernierEtatBoutons** pour ne traiter que les changements effectifs. Chaque fonction du commodo est associée à un bit spécifique via des masques prédéfinis : **MASK_VEILLEUSE** (bit 0), **MASK_PHARE** (bit 2), **MASK_CLIGN_D** (bit 5), etc., permettant une identification claire des commandes.

```

33 //===== MASQUES GP0 à GP7 =====
34
35 #define MASK_VEILLEUSE (1 << 0)
36 #define MASK_WARNING (1 << 1)
37 #define MASK_PHARE (1 << 2)
38 #define MASK_CODE (1 << 3)
39 #define MASK_CLIGN_G (1 << 4)
40 #define MASK_CLIGN_D (1 << 5)
41 #define MASK_STOP (1 << 6)
42 #define MASK_KLAXON (1 << 7)

```

Lorsqu'un changement d'état est détecté, la fonction **envoyerCommandeFeux()** génère et envoie une trame IM (*Input Message* - identifiant **0x05081F00**) contenant la nouvelle configuration des feux à appliquer via le registre GPLAT (adresse 0x1E). Un cas particulier est implémenté pour la gestion du klaxon, matérialisé par un buzzer connecté à la broche D3 (configurée en sortie PWM), qui est activé (état HIGH) lorsque le bit **MASK_KLAXON** (bit 7) est actif et désactivé (état LOW) lorsque le bouton est relâché. Pour le débogage et le suivi de l'état du système.

```

77 void traiterTrameCAN(struct can_frame trame) {
78     uint32_t id = trame.can_id & 0xFFFFFFF;
79
80     // ---- Trame OM
81     if (id == ID_OM_COMMODO && trame.can_dlc >= 2) {
82         uint8_t etatBrut = trame.data[1];
83         uint8_t boutons = ~etatBrut;
84
85         if (boutons != dernierEtatBoutons) {
86             dernierEtatBoutons = boutons;
87             envoyerCommandeFeux(boutons);
88             afficherEtatFeux(boutons);
89
90             // Gestion du buzzer selon GP7
91             if (boutons & MASK_KLAXON) {
92                 digitalWrite(BUZZER_PIN, HIGH); // Active le buzzer
93             } else {
94                 digitalWrite(BUZZER_PIN, LOW); // Désactive le buzzer
95             }
96         }
97     }

```

La fonction **afficherEtatFeux()** fournit une sortie claire dans le moniteur série, indiquant l'état de chaque fonction (veilleuse, phares, clignotants, etc.).

```

126 // ===== AFFICHAGE SERIAL =====
127
128 void afficherEtatFeux(uint8_t boutons) {
129     Serial.println("===== ÉTAT DES FEUX =====");
130     Serial.print("Clignotant Gauche: "); Serial.println(boutons & MASK_CLIGN_G ? "ACTIF" : "inactif");
131     Serial.print("Clignotant Droit : "); Serial.println(boutons & MASK_CLIGN_D ? "ACTIF" : "inactif");
132     Serial.print("Stop : "); Serial.println(boutons & MASK_STOP ? "ACTIF" : "inactif");
133     Serial.print("Klaxon : "); Serial.println(boutons & MASK_KLAXON ? "ACTIF" : "inactif");
134     Serial.print("Veilleuse : "); Serial.println(boutons & MASK_VEILLEUSE ? "ACTIF" : "inactif");
135     Serial.print("Phare : "); Serial.println(boutons & MASK_PHARE ? "ACTIF" : "inactif");
136     Serial.print("Code : "); Serial.println(boutons & MASK_CODE ? "ACTIF" : "inactif");
137     Serial.print("Warning : "); Serial.println(boutons & MASK_WARNING ? "ACTIF" : "inactif");
138     Serial.println("===== \n");
139 }

```

Chaque commande envoyée au module déclenche en retour une trame AIM (*Acknowledge Input Message* - identifiant **0x05200000**) qui sert d'accusé de réception. Ce mécanisme est géré via les variables **ackPending** et **ackPrinted**, assurant ainsi une communication fiable sans doublons. La gestion des temporisations et la vérification des timeout complètent ce système pour garantir une robustesse optimale. Cette architecture, combinant gestion par interruption, traitement conditionnel optimisé et validation systématique des commandes. L'ensemble du système démontre comment transformer des entrées utilisateur simples en commandes complexes tout en maintenant une intégrité et une sécurité opérationnelle maximales.

Description de l'implémentation avec Arduino

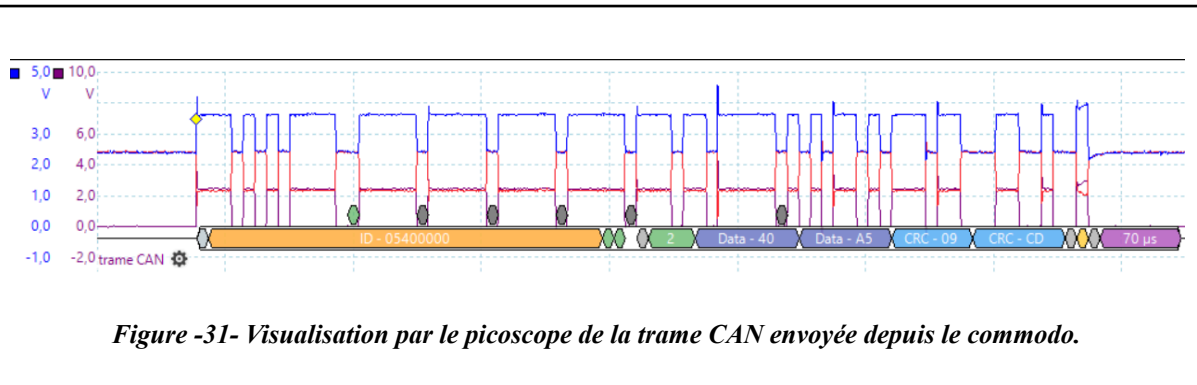
L'Arduino reçoit les trames OM émises par le *MCP25050* du commodo lorsqu'un bouton change d'état. Une interruption matérielle sur la broche INT du *MCP2515* signale l'arrivée de chaque trame. Le code lit l'état binaire des GP, applique l'inversion logique (appui = 1), et envoie des trames IM vers les modules feux correspondants.

Le TP inclut aussi l'intégration d'un buzzer Grove v1.2 pour le klaxon (activation sur BP4).

Résultats obtenus

La réactivité est très bonne, avec un délai quasi imperceptible entre l'appui sur le commodo et la réaction des feux/buzzer.

On visualise la trame OM lors de l'appui simultané sur les boutons GP1 (Warning), GP3 (Code), GP4 (Clignotant Gauche) et GP6 (Stop), le circuit MCP25050 génère une trame OM avec l'identifiant standard **0x05400000**, contenant dans son deuxième octet (**data[1]**) la valeur hexadécimale **0xA5**, soit le motif binaire 10100101 où les bits 1, 3, 4 et 6 positionnés à 0 indiquent les boutons actionnés selon la logique active-bas.



Ce signal est transmis via le bus CAN jusqu'au contrôleur *MCP2515* qui, après réception, déclenche une interruption matérielle sur l'Arduino. Le microcontrôleur procède alors à une inversion logique complète des bits via l'opérateur NOT (~), transformant ainsi 10100101 (**0xA5**) en 01011010 (**0x5A**).

Cette valeur inversée est ensuite analysée à l'aide d'une série de masques binaires soigneusement définis:

```
#define MASK_WARNING (1 << 1) // 00000010 (bit 1)
```

```
#define MASK_CODE (1 << 3) // 00001000 (bit 3)
```

```
#define MASK_CLIGN_G (1 << 4) // 00010000 (bit 4)
```

```
#define MASK_STOP (1 << 6) // 01000000 (bit 6)
```

Ces masques permettent d'isoler et d'identifier chaque fonction individuellement grâce à des opérations ET logique bit à bit. Par exemple, l'évaluation de (**0x5A & MASK_WARNING**) retourne une valeur non nulle, confirmant l'activation du Warning. Ces vérifications successives conduisent à la génération d'une trame IM contenant les commandes finales à envoyer au registre GPLAT du module d'éclairage, le Serial Monitor affiche l'état actualisé de chaque fonction grâce à une série de **Serial.print()** conditionnels qui interprètent la valeur **0x5A**, montrant clairement "**ACTIF**" pour les quatre fonctions concernées.

```

126 // ===== AFFICHAGE SERIAL =====
127
128 void afficherEtatFeux(uint8_t boutons) {
129     Serial.println("===== ÉTAT DES FEUX =====");
130     Serial.print("Clignotant Gauche: "); Serial.println(boutons & MASK_CLIGN_G ? "ACTIF" : "inactif");
131     Serial.print("Clignotant Droit: "); Serial.println(boutons & MASK_CLIGN_D ? "ACTIF" : "inactif");
132     Serial.print("Stop: "); Serial.println(boutons & MASK_STOP ? "ACTIF" : "inactif");
133     Serial.print("Klaxon: "); Serial.println(boutons & MASK_KLAXON ? "ACTIF" : "inactif");
134     Serial.print("Veilleuse: "); Serial.println(boutons & MASK_VEILLEUSE ? "ACTIF" : "inactif");
135     Serial.print("Phare: "); Serial.println(boutons & MASK_PHARE ? "ACTIF" : "inactif");
136     Serial.print("Code: "); Serial.println(boutons & MASK_CODE ? "ACTIF" : "inactif");
137     Serial.print("Warning: "); Serial.println(boutons & MASK_WARNING ? "ACTIF" : "inactif");
138     Serial.println("===== \n");
139 }

```

Visualisation de l'état des feux sur le serial monitor d'ArduinoIDE:

```
-----
[TX] Feux activés : 0x5A (0b1011010)
===== ÉTAT DES FEUX =====
Clignotant Gauche: ACTIF
Clignotant Droit : inactif
Stop           : ACTIF
Klaxon         : inactif
Veilleuse      : inactif
Phare          : inactif
Code           : ACTIF
Warning        : ACTIF
=====
[RX] AIM reçu : 0x0 (0b0)
-----
```

VI.3. TP3 : Vérifier le fonctionnement d'un bloc optique

Objectif

Le TP3 consiste à envoyer une séquence cyclique d'activation des différents feux du *VMD* et à vérifier, par lecture des trames de retour, que l'ordre est correctement exécuté par les modules cibles. Cet exercice combine :

- L'envoi de trames IM vers les blocs optiques.
- L'interrogation par trame IRM.
- La réception et l'analyse des trames OM de retour d'état.

L'intérêt est de valider que la commande émise correspond bien à l'état réel du module, et de mettre en évidence le mécanisme de supervision sur un bus **CAN**.

Cahier des charges:

On souhaite en enchaînement cyclique des différents états d'un bloc optique (Rien, Veilleuse, Veilleuse + Code, Veilleuse + Phare etc...) et un fonctionnement du clignoteur.

On réalisera en plus des fonctions de contrôle:

- On vérifiera que le module commandé renvoi bien une trame d'acquiescement.
- On vérifiera (par fonction logicielle) que les lampes commandées sont effectivement allumées (consommement du courant).
- On affichera quelles sont les lampes concernées ainsi que le résultat du test.
- Le clignotement (du clignotant) sera indépendant de la permutation des autres lampes ainsi que de la période de contrôle.

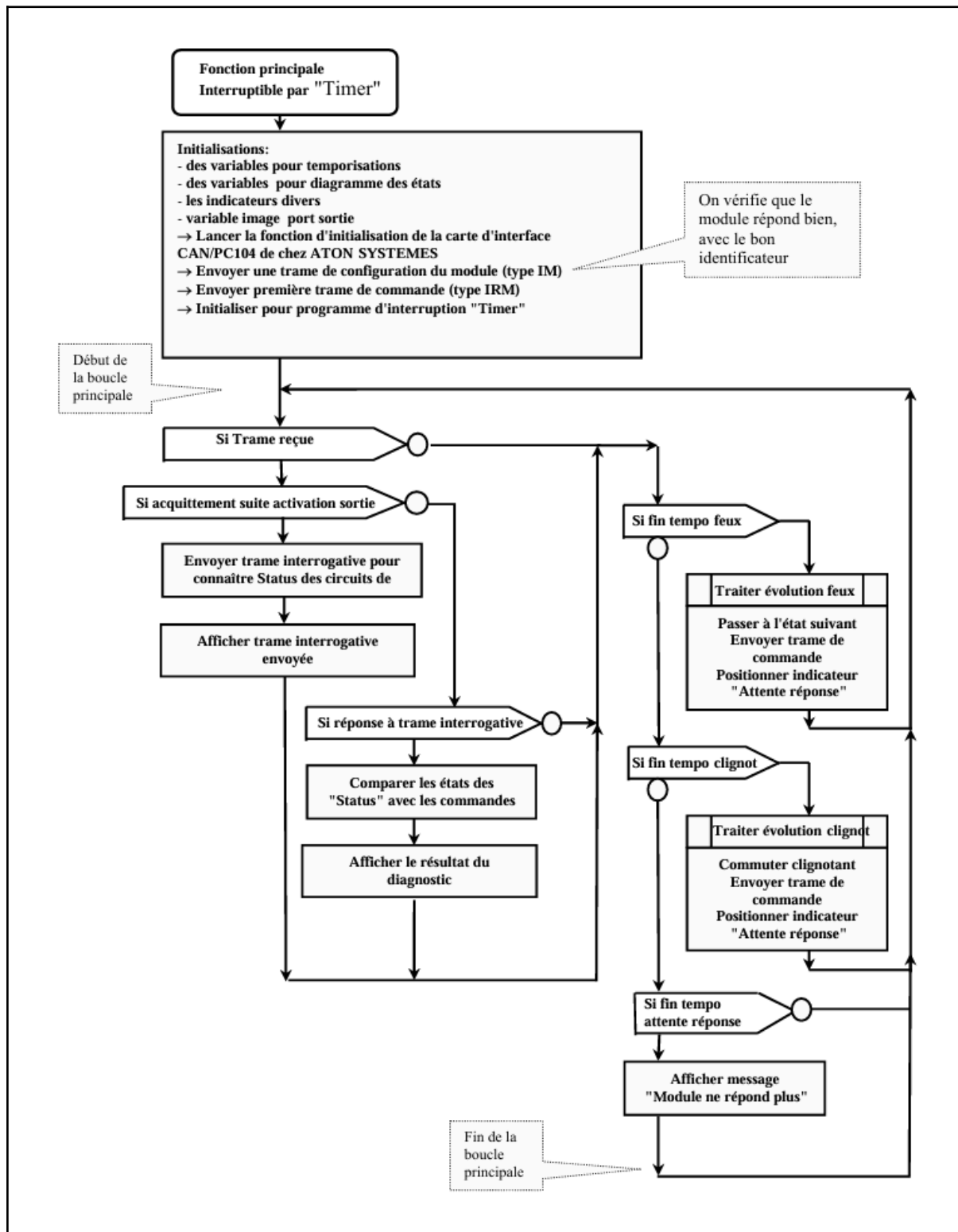
On impose les périodes suivantes, dans l'ordre de priorité décroissante:

- période de permutation des lampes 3,5 S.
- période de commutation du clignoteur 1,6 S.

Ces périodes devront pouvoir être modifiées facilement.

- Le programme devra permettre un changement aisé de bloc cible.

Organigramme:



Fonctionnement:

Le TP3 marque une étape importante dans la complexité en introduisant la gestion simultanée de plusieurs temporisations indépendantes et la vérification du bon fonctionnement des ampoules. Ce programme simule fidèlement le comportement d'un système d'éclairage automobile complet avec ses contraintes temporelles spécifiques. La structure de données commandes, utilisant des champs de bits (*bitfields*), est un choix technique remarquable : elle permet de stocker de manière compacte l'état de quatre feux différents (veilleuse, code, phare, clignotant droit) dans un seul octet, tout en offrant une interface de programmation claire via des noms symboliques. Les deux temporisations principales (**TEMPO_FEUX** à 3,5 s et **TEMPO_CLIGNOT** à 1,6 s) sont gérées via des comparaisons avec `millis()`, évitant ainsi les blocages tout en maintenant une précision acceptable pour ce type d'application.

```
29 #define TEMPO_FEUX      3500      // Durée entre états de feux (ms)
30 #define TEMPO_CLIGNOT   1600      // Fréquence clignotant droit (ms)
```

```
192 void loop() {
193     unsigned long now = millis();
194
195     if (now - t_prev_feux >= TEMPO_FEUX) {
196         t_prev_feux = now;
197         miseAJourFeux();
198     }
199
200     if (now - t_prev_cligno >= TEMPO_CLIGNOT) {
201         t_prev_cligno = now;
202         miseAJourClignotant();
203     }
204 }
```

La machine à états implémentée dans `miseAJourFeux()` suit un schéma classique de type "cycle through sequence" où chaque état active une combinaison particulière de feux.

```

130 //===== MISE À JOUR DE LA SÉQUENCE DE FEUX =====
131
132 void miseAJourFeux() {
133     static uint8_t etape = 0;
134     commandes = {0, 0, 0, commandes.clign_d}; // Reset sauf clignotant
135
136     switch (etape) {
137         case 1: commandes.veilleuse = 1; break;
138         case 2: commandes.veilleuse = commandes.code = 1; break;
139         case 3: commandes.veilleuse = commandes.phare = 1; break;
140         default: break; // Étape 0 : tout éteint
141     }
142
143     etape = (etape + 1) % 4;
144
145     uint8_t val = (commandes.clign_d << 3) | (commandes.phare << 2) |
146                 (commandes.code << 1) | commandes.veilleuse;
147
148     envoyerCommande(val);
149     afficherEtatFeux();
150 }

```

L'état du clignotant, quant à lui, est géré séparément dans `miseAJourClignotant()`, illustrant le principe de séparation des préoccupations - chaque fonction a une responsabilité unique et bien définie.

```

152 //===== CLIGNOTANT DROIT (AUTONOME) =====
153
154 void miseAJourClignotant() {
155     clignotant_on = !clignotant_on;
156     commandes.clign_d = clignotant_on;
157
158     uint8_t val = (commandes.clign_d << 3) | (commandes.phare << 2) |
159                 (commandes.code << 1) | commandes.veilleuse;
160
161     envoyerCommande(val);
162     afficherEtatFeux();
163 }

```

La fonction `envoyerCommande()` introduit une nouveauté importante : un mécanisme de timeout qui attend pendant 500 ms maximum un acquittement (ACK) avant de signaler une erreur. Cette temporisation permet de détecter les pannes de communication sans bloquer indéfiniment le système.

```

87 //===== ENVOI DE COMMANDE AVEC GESTION D'ACK =====
88
89 bool envoyerCommande(uint8_t val) {
90     struct can_frame frame;
91     frame.can_id = ID_IM_BLOC | CAN_EFF_FLAG;
92     frame.can_dlc = 3;
93     frame.data[0] = REG_GPLAT;
94     frame.data[1] = MASK_BLOC;
95     frame.data[2] = val;
96
97     ackRecu = false;
98     attenteAck = true;
99
100     if (mcp2515.sendMessage(&frame) != MCP2515::ERROR_OK) {
101         Serial.println("[ERREUR] Échec d'envoi CAN");
102         return false;
103     }
104
105     // Attend l'interruption ou timeout
106     unsigned long t0 = millis();
107     while (attenteAck && (millis() - t0 < 500));
108
109     if (ackRecu) return true;
110
111     Serial.println("[TIMEOUT] Aucun ACK reçu");
112     return false;
113 }
114

```

L'affichage détaillé via `afficherEtatFeux()` sert non seulement au débogage mais aussi à valider le bon fonctionnement global en montrant l'état de chaque feu à tout moment (fournit en temps réel sur le port série une visualisation détaillée de l'état de chaque feu, incluant leur représentation binaire et hexadécimale). La gestion des interruptions permet de filtrer spécifiquement les trames AIM en vérifiant leur identifiant avant de mettre à jour les drapeaux `ackRecu` et `attenteAck`.

```

69 //===== AFFICHAGE DÉTAILLÉ SUR MONITEUR =====
70
71 void afficherEtatFeux() {
72     Serial.println("\n[ÉTAT DES FEUX - " NOM_BLOC " ]");
73     Serial.print(" Veilleuse : "); Serial.println(commandes.veilleuse ? "ON" : "OFF");
74     Serial.print(" Code : "); Serial.println(commandes.code ? "ON" : "OFF");
75     Serial.print(" Phare : "); Serial.println(commandes.phare ? "ON" : "OFF");
76     Serial.print(" Clignotant: "); Serial.println(commandes.clign_d ? "ON" : "OFF");
77
78     uint8_t val = (commandes.clign_d << 3) | (commandes.phare << 2) |
79                 (commandes.code << 1) | commandes.veilleuse;
80
81     Serial.print(" Valeur binaire : 0b");
82     for (int8_t i = 3; i >= 0; i--) Serial.print((val >> i) & 1);
83     Serial.print(" (0x"); Serial.print(val, HEX); Serial.println(")");
84     Serial.println("-----");
85 }

```

Résultats obtenus

La séquence est stable et la vérification fonctionne correctement, ce qui assure que le bloc optique exécute bien la commande reçue.

La trame ci-dessus représente la trame IM envoyé vers le bloc optique “Feux avant droit” afin d'allumer les led suivantes :

- Le clignotant droit (GP3)
- Le code (GP1)
- La Veilleuse (GP0)
 - La valeur équivalente en binaire est de: 1011 (B en Hexadécimal) et donc 0 pour Gp4 a Gp7 c’est pour ca la valeur final decode est “0B”

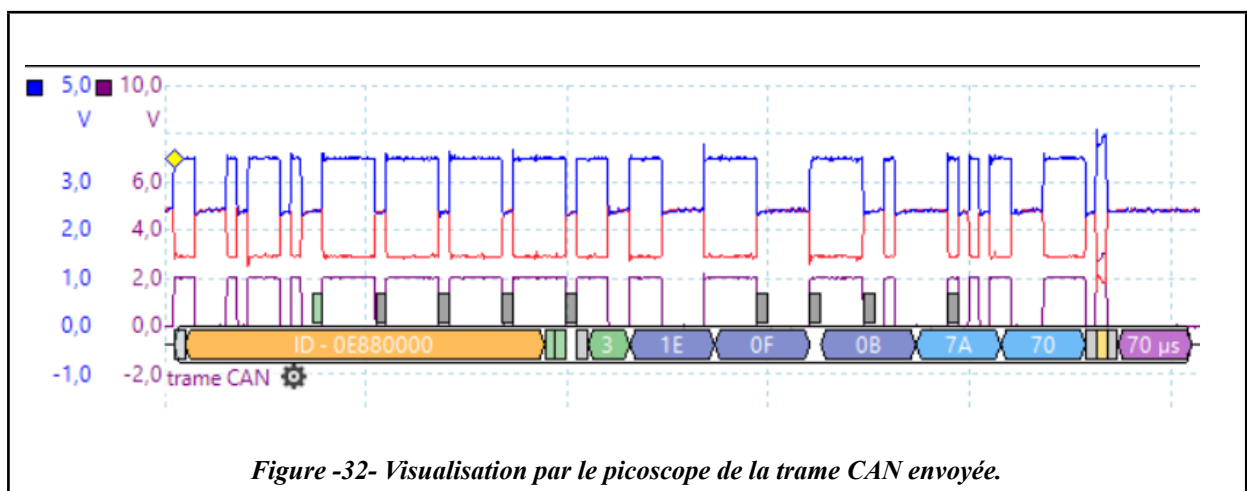


Figure -32- Visualisation par le picoscope de la trame CAN envoyée.

On visualise également un message de retour (id: **0x0EA00000** ⇒ bloc optique feux avant droit) signalant que la commande a bien été reçue et traitée et confirme la bonne réception de la commande.

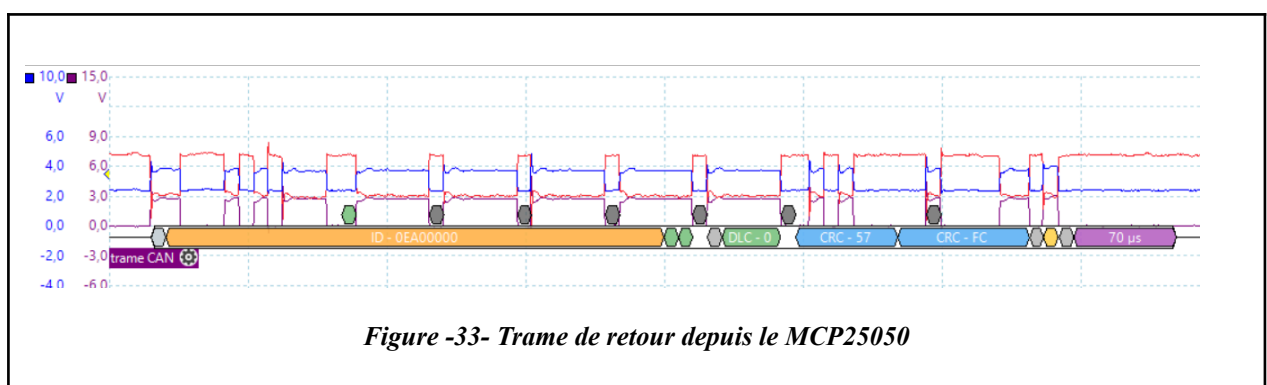


Figure -33- Trame de retour depuis le MCP25050

Visualisation de l'état des feux sur le serial monitor d'ArduinoIDE:

```
[ACK] Reçu de 0xEA00000 → Valeur : 0x3  
  
[ÉTAT DES FEUX - FVD]  
Veilleuse : ON  
Code      : ON  
Phare     : OFF  
Clignotant: ON  
Valeur binaire : 0b1011 (0xB)
```

VI.4. TP4 : Commander feux a partir du commodo feux

Objectif

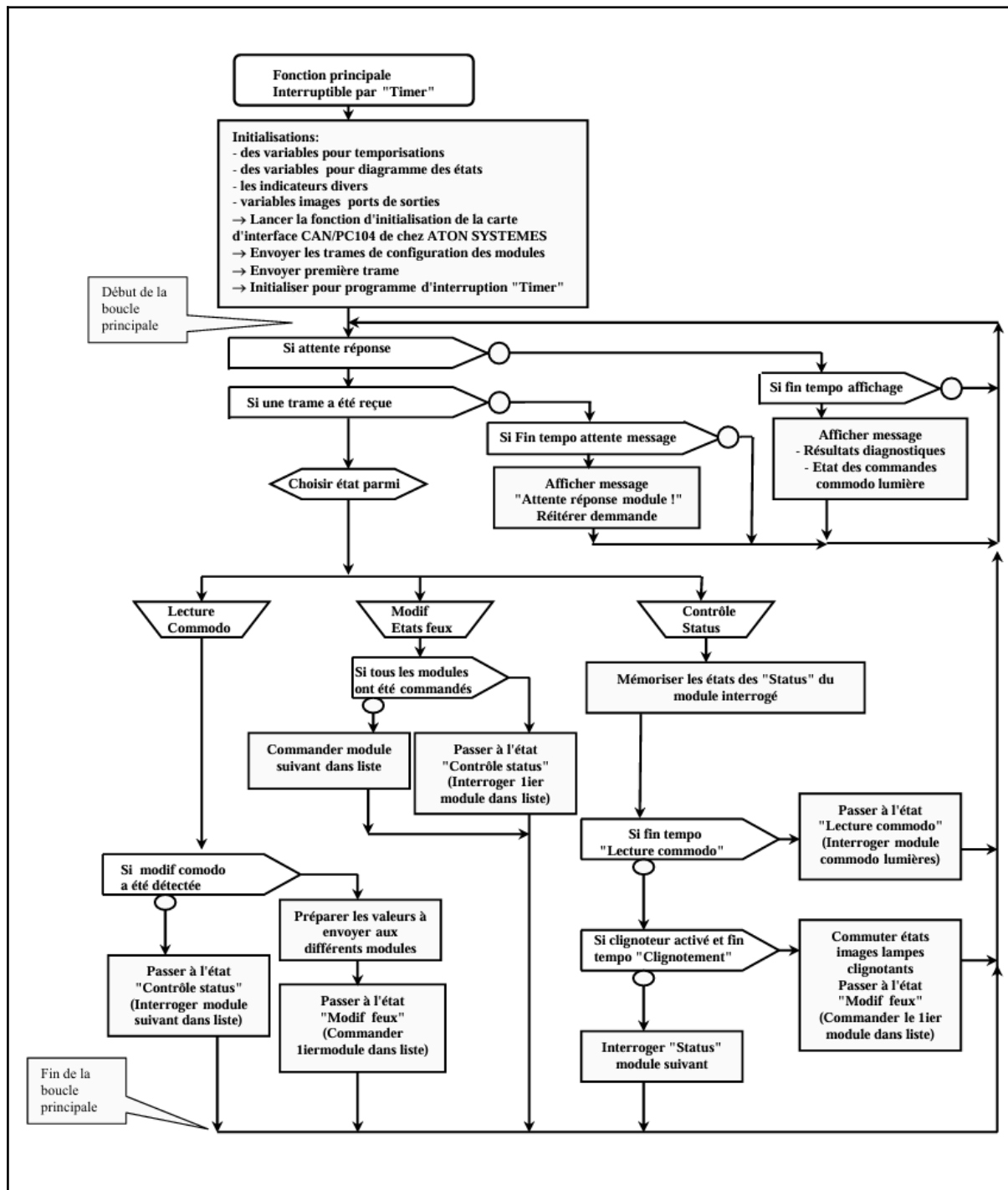
Le TP4 met en œuvre une activation ciblée des feux uniquement lorsqu'un bouton spécifique du commodo est pressé, avec ajout de comportements temporisés comme le clignotement. Ce TP illustre la mise en place de logiques conditionnelles et de séquences temporisées côté Arduino, en fonction des signaux reçus du *MCP25050*.

Cahier des charges:

A intervalles de temps réguliers, on interroge le module sur lequel est connecté le commodo lumières afin de connaître son état. En fonction de l'état du commodo lumière, on active les différentes lampes des blocs optiques avant et arrière.

- La temporisation nécessaire au fonctionnement des clignotants est réalisée par "Timer programmable" (interne au micro-processeur).
- Les différentes commandes imposées par la position de la manette commodo seront affichées individuellement.
- On contrôle le bon fonctionnement des ampoules des différents blocs optiques.

Organigramme:



Fonctionnement:

Le TP4 représente l'aboutissement des travaux précédents en intégrant toutes les fonctionnalités dans un système cohérent de gestion des feux basé sur les entrées du commodo. Le système repose sur plusieurs couches fonctionnelles clairement définies, la couche bas niveau (*initCAN()*) initialise le contrôleur *MCP2515* avec une configuration (100kbps, mode normal)

```
75 //===== INITIALISATION CAN ET MODULES =====
76
77 void initCAN() {
78     SPI.begin();
79     mcp2515.reset();
80     mcp2515.setBaudrate(CAN_SPEED, CAN_CLOCK);
81     mcp2515.setNormalMode();
82 }
```

tandis que *configCommodo()* et *configFeux()* configurent respectivement les modules en entrées (0xFF) et sorties (0x0F pour les bits 0-3).

```
84 void configCommodo() {
85     struct can_frame frame = { .can_id = ID_IM_COMMODO | CAN_EFF_FLAG, .can_dlc = 3 };
86     frame.data[0] = REG_GPDDR; frame.data[1] = 0xFF; frame.data[2] = 0xFF; // Tout en entrée
87     mcp2515.sendMessage(&frame);
88     delay(CAN_SEND_DELAY);
89
90     frame.data[0] = REG_IOTEN; frame.data[2] = 0xFF; // Active interruptions
91     mcp2515.sendMessage(&frame);
92     delay(CAN_SEND_DELAY);
93 }
```

```
95 void configFeux(uint32_t id) {
96     struct can_frame frame = { .can_id = id | CAN_EFF_FLAG, .can_dlc = 3 };
97     frame.data[0] = REG_GPDDR; frame.data[1] = 0x0F; frame.data[2] = 0xF0; // Sorties bits 3-0
98     mcp2515.sendMessage(&frame);
99     delay(CAN_SEND_DELAY);
100
101     frame.data[0] = REG_GPLAT; frame.data[2] = 0x00; // Éteint les feux
102     mcp2515.sendMessage(&frame);
103     delay(CAN_SEND_DELAY);
104 }
```


La structure **EtatCommodo** sert de modèle de données central, encapsulant l'état complet du système avec des champs spécifiques pour chaque fonction (veilleuse, phares, clignotants, etc.). La fonction **updateFeux()**, cœur de la logique métier, combine les états des commandes avec la temporisation des clignotants (gérée par **blinkState** et **BLINK_INTERVAL**) tout en respectant les priorités système (ex: le warning prime sur les clignotants).

```

122 void updateFeux() {
123     // Clignotement ON/OFF automatique
124     if (millis() - tlastBlink > BLINK_INTERVAL) {
125         blinkState = !blinkState;
126         tlastBlink = millis();
127     }
128
129     // Réinitialisation
130     etat.fad = etat.fag = etat.ard = etat.arg = 0;
131
132     // Veilleuses avant (bit 0), arrière (bit 3)
133     if (etat.veilleuse) {
134         etat.fag |= 0x01; etat.fad |= 0x01;
135         etat.arg |= 0x08; etat.ard |= 0x08;
136     }
137
138     // Warning prioritaire sur les clignotants
139     if (etat.warning) {
140         etat.clignG = etat.clignD = true;
141     }
142
143     // Code et Phare
144     if (etat.code) { etat.fag |= LED_CODE; etat.fad |= LED_CODE; }
145     if (etat.phare) { etat.fag |= LED_PHARE; etat.fad |= LED_PHARE; }
146
147     // Clignotants clignent si état actif
148     if (etat.clignG && blinkState) {
149         etat.fag |= LED_CLIGN_AV; etat.arg |= LED_CLIGN_AR;
150     }
151     if (etat.clignD && blinkState) {
152         etat.fad |= LED_CLIGN_AV; etat.ard |= LED_CLIGN_AR;
153     }
154
155     // Stop actif
156     if (etat.stop) {
157         etat.ard |= LED_STOP_CMD; etat.arg |= LED_STOP_CMD;
158     }
159
160     // Klaxon arrière gauche
161     if (etat.klaxon) {
162         etat.arg |= LED_KLAXON_CMD;
163     }

```

Chaque modification d'état déclenche un envoi **CAN** via **envoyerFeux()** qui applique un masque strict (**0x0F**) pour ne modifier que les bits 0-3 des registres GPLAT, avec un contrôle de débit intégré (**CAN_SEND_DELAY**) pour éviter la saturation du bus.

```

165 // Envoi des états finaux aux modules (avec délai intégré dans envoyerFeux())
166 envoyerFeux(ID_IM_FAD, etat.fad);
167 envoyerFeux(ID_IM_FAG, etat.fag);
168 envoyerFeux(ID_IM_ARD, etat.ard);
169 envoyerFeux(ID_IM_ARG, etat.arg);
170 }

```

La réception des données du commodo est gérée par **recevoirEtatCommodo()** qui interprète les trames OM en appliquant la logique inverse (active-low) et met à jour le modèle de données.

```

197 //===== TRAITEMENT DES MESSAGES CAN ENTRANTS =====
198
199 void recevoirEtatCommodo(struct can_frame f) {
200     if ((f.can_id & 0x1FFFFFFF) != ID_OM_COMMODO || f.can_dlc < 2) return;
201     uint8_t p = f.data[1];
202
203     etat.veilleuse = !(p & MASK_VEILLEUSE);
204     etat.warning   = !(p & MASK_WARNING);
205     etat.phare     = !(p & MASK_PHARE);
206     etat.code      = !(p & MASK_CODE);
207     etat.clignG    = !(p & MASK_CLIGN_G);
208     etat.clignD    = !(p & MASK_CLIGN_D);
209     etat.stop      = !(p & MASK_STOP);
210     etat.klaxon    = !(p & MASK_KLAXON);
211
212     // On ne met pas à jour les feux immédiatement pour éviter les rafales
213     // La mise à jour sera gérée par la boucle principale avec le bon timing
214 }

```

Un système de polling optimisé (**demanderEtat()**) interroge périodiquement le commodo (**POLLING_INTERVAL**) sans bloquer le système grâce à l'utilisation de **millis()**.

```

218 void demanderEtat() {
219     // Respecte le délai minimum entre envois CAN
220     if (millis() - tLastCanSend >= CAN_SEND_DELAY) {
221         struct can_frame frame = { .can_id = ID_IRM_COMMODO | CAN_EFF_FLAG, .can_dlc = 1 };
222         frame.data[0] = REG_GPLAT; // Lire l'état des entrées
223         mcp2515.sendMessage(&frame);
224         tLastPoll = millis();
225         tLastCanSend = millis();
226     }
227 }

253 // Requête périodique d'état
254 if (millis() - tLastPoll >= POLLING_INTERVAL) {
255     demanderEtat();
256 }

```

L'affichage série (**afficherEtat()**), actualisé selon **DISPLAY_INTERVAL**, fournit une visibilité complète sur l'état du système en utilisant **memcmp()** pour détecter les changements. La boucle principale orchestre l'ensemble de ces composants de manière non-bloquante, respectant scrupuleusement les différentes temporisations (**FEUX_UPDATE_INTERVAL**) tout en maintenant une architecture modulaire, fiable et parfaitement adaptée aux contraintes du système.

```

172 //===== AFFICHAGE SERIAL DE L'ÉTAT DU COMMODO =====
173
174 void afficherEtat() {
175     if (memcmp(&etat, &dernierEtat, sizeof(EtatCommodo)) != 0 || millis() - tlastDisplay > DISPLAY_INTERVAL) {
176         Serial.println("\n=== ETAT COMMODO ===");
177         Serial.print("Clign. G : "); Serial.println(etat.clignG ? "Actif" : "Inactif");
178         Serial.print("Clign. D : "); Serial.println(etat.clignD ? "Actif" : "Inactif");
179         Serial.print("STOP : "); Serial.println(etat.stop ? "Actif" : "Inactif");
180         Serial.print("KLAXON : "); Serial.println(etat.klaxon ? "Actif" : "Inactif");
181         Serial.print("Veilleuse : "); Serial.println(etat.veilleuse ? "Actif" : "Inactif");
182         Serial.print("Warning : "); Serial.println(etat.warning ? "Actif" : "Inactif");
183         Serial.print("Phare : "); Serial.println(etat.phare ? "Actif" : "Inactif");
184         Serial.print("Code : "); Serial.println(etat.code ? "Actif" : "Inactif");
185
186         Serial.println("=== ETAT FEUX ===");
187         Serial.print("FAD : 0x"); Serial.println(etat.fad, HEX);
188         Serial.print("FAG : 0x"); Serial.println(etat.fag, HEX);
189         Serial.print("ARD : 0x"); Serial.println(etat.ard, HEX);
190         Serial.print("ARG : 0x"); Serial.println(etat.arg, HEX);
191
192         memcpy(&dernierEtat, &etat, sizeof(EtatCommodo));
193         tlastDisplay = millis();
194     }
195 }

```

```

258 // Mise à jour des feux avec contrôle de timing
259 if (millis() - tLastFeuxUpdate >= FEUX_UPDATE_INTERVAL) {
260     updateFeux();
261     afficherEtat(); // On affiche l'état seulement quand on met à jour les feux
262     tLastFeuxUpdate = millis();
263 }
264 }

```

Chaque fonction a été optimisée pour minimiser l'utilisation des ressources tout en garantissant un comportement temps-réel précis..

Description de l'implémentation avec Arduino

Le programme lit l'état du commodo via interruption, et selon le bouton actif, envoie la trame IM correspondante au bloc optique.

Résultats obtenus

Les fonctions sont conformes aux attentes, le clignotement est fluide.

Les captures *PicoScope* mettent en évidence deux états distincts du système de gestion des feux. Dans le premier cas d'activation, on observe clairement la séquence de commandes CAN lorsque les phares avant et feux stop arrière sont activés via le commodo, chaque module reçoit sa trame spécifique

- **0x0E880000** pour feux avant droit et **0x0E080000** pour feux avant gauche avec la valeur à écrire dans le registre: **0x04** soit 0100 en binaire activant GP2
- **0x0F880000** pour feux arrière droit et **0x0F080000** pour feux arrière gauche la valeur à écrire dans le registre: **0x02** soit 0010 en binaire activant GP2

suivie systématiquement d'un accusé de réception (AIM) correspondant, confirmant que chaque commande a bien été exécutée.

Packet	Start Time	End Time	ID	RTR	FDL	DLC	Data	CRC	ACK	CRC Valid
1	-359.8 ns	949.4 µs	0E880000	0	0	3	1E 0F 04	5D 8A	0	✓
2	999.7 µs	1.7 ms	0EA00000	0	0	0		57 FC	0	✓
3	5.129 ms	6.099 ms	0E080000	0	0	3	1E 0F 04	0A BA	0	✓
4	6.148 ms	6.848 ms	0E200000	0	0	0		30 14	0	✓
5	10.26 ms	11.23 ms	0F880000	0	0	3	1E 0F 02	20 17	0	✓
6	11.28 ms	11.98 ms	0FA00000	0	0	0		5D B5	0	✓
7	15.39 ms	16.34 ms	0F080000	0	0	3	1E 0F 02	77 27	0	✓
8	16.39 ms	17.08 ms	0F200000	0	0	0		3A 5D	0	✓
9	31.4 ms	32.17 ms	05041E07	0	0	1	1E	33 92	0	✓

Le second cas montre l'état inactif du système où toutes les trames IM contiennent la valeur **0x00** dans le champ Data, éteignant ainsi tous les feux, avec là encore une confirmation par trame AIM pour chaque module. Ces échanges démontrent la robustesse de la communication CAN, où chaque commande vers les registres GPLAT (adresse **0x1E**) avec masque **0x0F** est parfaitement acquittée, validant à la fois l'intégrité des données via les CRC et la bonne réception par les modules. La structure cohérente des trames (ID spécifique à chaque module, donnée positionnant les bits correspondants aux fonctions désirées) et la séquence invariable commande/accusé illustrent le fonctionnement fiable et prévisible de l'architecture embarquée, parfaitement conforme aux spécifications techniques du système et au code source implémenté.

Packet	Start Time	End Time	ID	RTR	FDL	DLC	Data	CRC	ACK	CRC Valid
1	60.19 ns	959.9 µs	0E880000	0	0	3	1E 0F 00	05 45	0	✓
2	1.01 ms	1.71 ms	0EA00000	0	0	0		57 FC	0	✓
3	5.139 ms	6.099 ms	0E080000	0	0	3	1E 0F 00	52 75	0	✓
4	6.149 ms	6.849 ms	0E200000	0	0	0		30 14	0	✓
5	10.27 ms	11.23 ms	0F880000	0	0	3	1E 0F 00	6E BC	0	✓
6	11.28 ms	11.98 ms	0FA00000	0	0	0		5D B5	0	✓
7	15.4 ms	16.36 ms	0F080000	0	0	3	1E 0F 00	39 8C	0	✓
8	16.41 ms	17.1 ms	0F200000	0	0	0		3A 5D	0	✓
9	20.45 ms	21.22 ms	05041E07	0	0	1	1E	33 92	0	✓

Visualisation de l'état des feux sur le serial monitor d'ArduinoIDE:

```

=== ETAT COMMODOS ===
Clign. G : Inactif
Clign. D : Inactif
STOP : Actif
KLAXON : Inactif
Veilleuse : Inactif
Warning : Inactif
Phare : Actif
Code : Inactif
=== ETAT FEUX ===
FAD : 0x4
FAG : 0x4
ARD : 0x2
ARG : 0x2

```

VII. Comparaison entre Arduino Uno R3 et EID210

Critère	EID210	Arduino Uno R3
Langage de programmation	Assembleur 68000 + C (toolchain fournie : éditeur, assembleur, cross-compileur C/C++ GNU, moniteur avec chargement S-Record)	C++ (framework Arduino), IDE simple, très large écosystème de bibliothèques
Interface CAN	Propriétaire via bus PC/104 + carte ATON CAN basée sur un contrôleur SJA1000 ; le VMD/CAN01A utilise des modules d'E/S MCP25050 et un transceiver de couche physique 82C251	SPI + contrôleur MCP2515 sur shield (transceiver MCP2551), approche standard « maker »
Facilité de développement	Moyenne: chaîne de compilation 68332, gestion PC/104, bibliothèques CAN en C ; apprentissage formateur mais plus « bas niveau »	Élevée: sketches, bibliothèques prêtes à l'emploi (MCP2515, etc.), déploiement USB direct
Coût	Élevé: architecture multi-cartes (EID210 + ATON CAN + modules VMD), format PC/104 et matériel didactique spécialisé	Faible: carte UNO + shield CAN grand public
Portabilité	Faible: « unité centrale programmable » en châssis, alim 12 V (jusqu'à 20 A dans la config VMD), câblage didactique de banc	Excellente: carte compacte, alimentée par USB, facilement embarquable

VIII. ANNEXE

1. Documents techniques et constructeurs:

- **Microchip :**
 - MCP2502X/5X CAN I/O Expander Family Data Sheet – MCP25020 / MCP25050: [MCP2502X/5X CAN I/O Expander Family Data Sheet](#)
 - MCP2515 Stand-Alone CAN Controller with SPI Interface Data Sheet: [MCP2515 Family Data Sheet](#)
 - MCP2551 High-Speed CAN Transceiver Data Sheet: [MCP2551 High-Speed CAN Transceiver](#)
- **Seeed Studio :**
 - Documentation CAN-BUS Shield v1.1 / v1.2: [CAN-BUS Shield V1.2 | Seeed Studio Wiki](#)
- **Arduino :**
 - Arduino Uno R3 Documentation officielle: [UNO R3 | Arduino Documentation](#)
- **Pico Technology :**
 - Documentation PicoScope 3000 Series (CAN decoding): [PicoScope 3000 Series USB3.0 oscilloscope specifications](#)

2. Documents pédagogiques et manuels du VMD:

- *Manuel des TP Véhicule Multiplexé Didactique (VMD) (document fourni)*
- *Spécifications VMD & module CAN01A (document fourni) ([Véhicule Multiplexé Didactique complet](#) / [Ensemble d'étude des Réseaux Locaux Industriels: BUS CAN](#))*
- *EID210 000 Guide Technique (document fourni, système EID210 + carte ATON CAN, PC/104, SJA1000) ([Carte d'étude du microprocesseur microcontrôleur 68332](#) / [Carte industrielle interface réseau CAN sur bus PC104](#))*

3. Sources externes utilisées pour enrichir certaines parties:

- **National Instruments (NI) :**
Controller Area Network (CAN) Protocol Overview: [Controller Area Network \(CAN\) Protocol Overview - NI](#)